Chapter 1
Introduction

1.1 Background

In today's rapidly evolving digital ecosystem, individuals and organizations increasingly rely on online platforms to store and manage sensitive information. Essential documents such as Aadhaar cards, property papers, bank statements, insurance policies, contracts, and legal records are now primarily stored in digital format. While digital transformation has enhanced accessibility and efficiency, it has also introduced significant concerns related to cybersecurity, unauthorized access, data privacy, and controlled information sharing.

Traditional methods of storing confidential data—such as physical lockers or basic cloud storage—lack intelligent access control and automated monitoring. Most cloud platforms rely solely on password-based protection, which makes them vulnerable to hacking, phishing attacks, password loss, or unauthorized sharing. Moreover, there is limited control over *conditional access*, meaning users cannot easily define rules about who can access their data and under what circumstances.

To address these challenges, the **Nominee-Based Access System (Next-Access)** is designed as a Secure Digital Vault that ensures encrypted storage and intelligent access management. Instead of relying on manual data sharing or emergency-based triggers, the system introduces a structured access-control framework where users can define trusted nominees with role-based permissions. Access is granted strictly under predefined verification protocols and user-defined rules.

The system integrates advanced cybersecurity mechanisms such as AES-256 encryption, multi-factor authentication (MFA), AI-based activity monitoring, and secure cloud architecture. AI continuously analyzes login behavior and access patterns to detect suspicious activity and prevent unauthorized access attempts. This proactive monitoring strengthens the system's defense against data breaches.

Unlike traditional storage systems, this platform focuses on controlled, rule-based, and secure document management. It ensures that digital assets remain confidential, accessible only to authorized individuals, and protected against misuse.

In summary, this project lies at the intersection of cybersecurity, artificial intelligence, and digital trust. It aims to create a secure, intelligent, and scalable digital vault system that enhances privacy, transparency, and controlled access management in the modern digital era.

1.2 Objectives

➤ **To Develop a Secure Digital Vault on Google Cloud Drive:**
The primary objective is to design a highly secure and encrypted digital vault where users can safely store sensitive documents such as Aadhaar cards, property papers, insurance policies, and legal agreements. The system must ensure confidentiality through AES-256 encryption and secure cloud storage.

➤ **To Implement Role-Based Nominee Access Control:**
The system aims to introduce a structured access mechanism where users can assign trusted nominees with predefined access levels (view-only, limited access, or full access). Access will be granted only after successful identity verification and according to user-defined permissions.

➤ **To Security Monitoring:**
Artificial Intelligence will be integrated to monitor user activity, detect suspicious login attempts, analyze behavior patterns, and generate security alerts. This ensures proactive protection against cyber threats and unauthorized access.

➤ **To Ensure Multi-Factor Authentication (MFA):**
The project aims to enhance security by implementing two-factor authentication (2FA), biometric verification (face recognition), and Code-based login mechanisms.

➤ **To Provide Cross-Platform Accessibility:**
The system will be developed using React Native and secure backend frameworks to ensure accessibility across Android and iOS devices while maintaining high performance and reliability.

1.3 Purpose, Scope & Applicability

1.3.1 Purpose

The purpose of this project is to provide a centralized, secure, and intelligent digital vault that enables users to store and manage sensitive documents with complete control over access permissions.

Instead of focusing on post-death access mechanisms, the system emphasizes controlled and secure document sharing during the user's lifetime. It enables users to grant conditional access to trusted individuals while maintaining strong authentication and encryption standards.

The platform enhances digital trust by ensuring:

- Confidential storage
- Controlled access
- AI-based threat detection
- Secure data sharing

This reduces the risks of data theft, unauthorized disclosure, and password mismanagement.

1.3.2 Scope

The scope of the Nominee-Based Access System includes:

- Development of a secure cloud-based encrypted digital vault
- Implementation of role-based access control (RBAC)
- Integration of AI-driven login behavior monitoring
- Multi-factor authentication (OTP + biometric verification)
- Secure document upload, encryption, storage, and retrieval

The system is scalable and can be expanded for enterprise-level applications. It can be adapted for use in:

- Banking and financial institutions
- Legal document management firms
- Corporate data protection systems
- Government digital record management

The project primarily focuses on software-based security architecture without hardware-based IoT components.

1.3.3 Applicability

The Nominee-Based Access System has broad applicability across multiple domains:

Individual Users: Individuals can securely store personal, financial, and legal documents and share them selectively with trusted contacts through verified access.

Corporate Sector: Organizations can use the platform to manage confidential internal documents with role-based access control and AI-based monitoring.

Legal & Financial Institutions: Law firms and insurance companies can securely manage client documents and control access permissions based on roles.

Government & Administrative Systems: The system can be adapted for secure citizen record management with strong authentication mechanisms.

By focusing on encryption, AI monitoring, and secure authentication, the system provides a flexible and scalable solution suitable for both personal and institutional use.

1.4 Achievement

The Nominee-Based Access System successfully achieves the following milestones:

- Implementation of AES-256 encrypted document storage
- Development of role-based nominee access control
- Integration of AI-based login and activity monitoring
- Multi-factor authentication (OTP + biometric verification)
- Secure cloud-based backend using Node.js/Django
- Cross-platform compatibility via React Native

The project demonstrates how modern cybersecurity principles and artificial intelligence can be combined to create a secure, intelligent, and scalable digital document management system.
By removing IoT dependency and post-death activation logic, the system becomes more practical, deployable, and legally adaptable across various sectors.

Chapter 2
Survey of Technologies

This Nominee-Based Access System(Next-Access) creates a secure digital vault for storing important personal and legal documents. It allows users to assign trusted nominees who can access the documents in emergencies or after the user's death, using biometric authentication and AI-based security features.

1. Programming Languages

- **React Native (Frontend):** React Native is employed to develop a cross-platform mobile application that works on both WebApplication. It provides a smooth, responsive user interface, as well as document management functionalities. The React Native ecosystem supports integration with device hardware like cameras and fingerprint sensors, enabling secure and user-friendly access to the vault.

- **Node.js (Backend):** The backend of the system is developed using either Node.js (JavaScript). These frameworks are chosen for their speed, security, and scalability. The backend handles core functions including user registration, login, nominee authorisation,. It manages communication between the mobile app, database, and AI models, ensuring smooth and secure operation of the system.

- **Java (Backend):** Java is used as the backend programming language due to its robustness, performance, and enterprise-level security features. It is responsible for implementing business logic, managing nominee access control, handling secure API requests, and processing user data. Java ensures high reliability and scalability of the system.

- **MongoDB (Database):** User data and sensitive documents are securely stored, which supports real-time updates, or MongoDB, which offers flexible document structure. Both databases incorporate encryption mechanisms to safeguard data integrity and confidentiality. This ensures that in the event of a data breach, the documents remain protected and unreadable without proper decryption keys.

2. Development Frameworks And IDEs

- **React.js:**
  React.js is used as the frontend framework to build a component-based and responsive user interface. It allows the development of reusable components such as login forms, dashboards, document upload modules, and nominee management panels. React ensures fast rendering through its Virtual DOM and provides seamless integration with authentication services and backend APIs.
- **Tailwind CSS:**
  Tailwind CSS is used for designing and styling the frontend interface. It is a utility-first CSS framework that enables rapid development of clean, modern, and responsive designs. Tailwind ensures mobile-first responsiveness and consistent styling throughout the application.
- **Spring Boot:**
  Spring Boot is used as the backend framework to develop RESTful APIs and manage

server-side operations. It simplifies backend configuration and integrates easily with security modules and databases. Spring Boot handles API routing, authentication validation, nominee access control, and secure communication between frontend and database layers.

- **Clerk (Authentication Service):**
  Clerk is used for secure user authentication and identity management. It manages user registration, login, session handling, and multi-factor authentication. Clerk generates JWT tokens that are validated by the Spring Boot backend to ensure secure and authorized access to protected resources.
- **IntelliJ IDEA:**
  IntelliJ IDEA is used as the primary Integrated Development Environment (IDE) for backend development. It provides advanced debugging tools, Spring Boot integration, dependency management support, and efficient code navigation, enhancing overall developer productivity.

3. Backend in Project

- **MongoDB:** MongoDB is used as the NoSQL database for storing user data, nominee details, and document metadata. Its flexible document-oriented structure allows efficient handling of complex data formats. MongoDB integrates seamlessly with Spring Boot and ensures secure storage of sensitive information.

- **JWT (JSON Web Token):** JWT is used for secure and stateless authentication between frontend and backend. After successful login through Clerk, a JWT token is generated and sent with API requests. The Spring Boot backend verifies this token before granting access to protected endpoints, ensuring secure communication.

- **Postman:** Postman is used for testing and debugging REST APIs during development. It helps verify authentication endpoints, test API responses, validate JWT tokens, and detect errors such as unauthorized or forbidden access responses.

- **Ngrok:** Ngrok is used to expose the local backend server to the internet during development. It enables secure tunneling, allowing webhook testing and external service integration while running the backend on a local machine.

4. Deployment Platforms

- **Vercel:** Vercel is used for deploying the React frontend application. It provides automatic deployment through GitHub integration, continuous integration and delivery (CI/CD), environment variable management, and global content delivery network (CDN) support for fast performance.

- **Backend Deployment (Spring Boot Server):** The Spring Boot backend can be deployed on cloud platforms such as Render, Railway, AWS, or VPS servers. These platforms provide scalability, secure hosting, and reliable production-level performance.

Chapter 3
Requirement and Analysis

3.1 Problem Definition

In today's digital environment, individuals store essential documents such as identification records, property papers, insurance policies, financial statements, and legal agreements in online platforms. Although digital storage has improved convenience, it also introduces serious security and access management challenges.

Many existing storage systems rely only on password-based protection, which creates risks such as forgotten credentials, unauthorized sharing, hacking attempts, and lack of structured access control. Additionally, there is no proper mechanism that allows controlled nominee-based access while maintaining confidentiality and system security.

Users often face the following issues:

- Important documents are scattered across multiple platforms.
- Password sharing compromises privacy and security.
- Unauthorized access attempts increase the risk of data theft.
- No structured role-based access control for trusted individuals.
- Manual document transfer methods are insecure and inefficient.

To address these challenges, the proposed **Nominee-Based Access System (Next-Access)** introduces a secure, centralized digital vault. The system allows users to upload and manage important documents securely while assigning trusted nominees with predefined access permissions.

The system ensures:

- Secure storage using encryption
- JWT-based authentication and authorization
- Role-based nominee access control
- Controlled API access via Spring Boot backend
- Secure identity verification using Clerk authentication

Unlike traditional storage systems, this platform emphasizes structured access management, strong authentication mechanisms, and modern cybersecurity practices without relying on IoT devices or automated death-trigger mechanisms.

3.2 Requirement Specification

The requirement specification defines the essential features and system expectations. It includes functional and non-functional requirements.

Functional Requirements

1. **User Registration and Authentication:** The system must allow users to register and log in securely using Clerk authentication. Multi-factor authentication and session management must be supported. JWT tokens are generated upon successful login and validated by the backend.

2. **Document Upload and Management:** Users should be able to upload, view, update, and delete documents. Each document must be securely stored, and metadata should be maintained in MongoDB. The system should support categorization and organized document management.

3. **Nominee Setup and Role-Based Access:** Users must be able to assign trusted nominees and define access levels such as view-only or limited access. Access is granted only after proper authentication and authorization checks via JWT validation.

4. **Secure API Communication:** All communication between frontend and backend must be protected through JWT-based authorization. Unauthorized API access must return appropriate HTTP responses such as 401 (Unauthorized) or 403 (Forbidden).

5. **Access Control Management:** The system must implement role-based access control (RBAC) to ensure that nominees can only access documents permitted by the user.

6. **Activity Logging and Monitoring:** All login attempts, document access requests, and nominee actions must be logged for auditing and security monitoring.

Non-Functional Requirements

1. **Security:** The system must ensure strong encryption, secure token validation, protected APIs, and data confidentiality.

2. **Performance:** API responses should be fast and optimized to ensure smooth frontend interaction.

3. **Scalability:** The system should handle increasing numbers of users and documents without performance degradation.

4. **Reliability:** Data should remain consistent and securely stored in MongoDB.

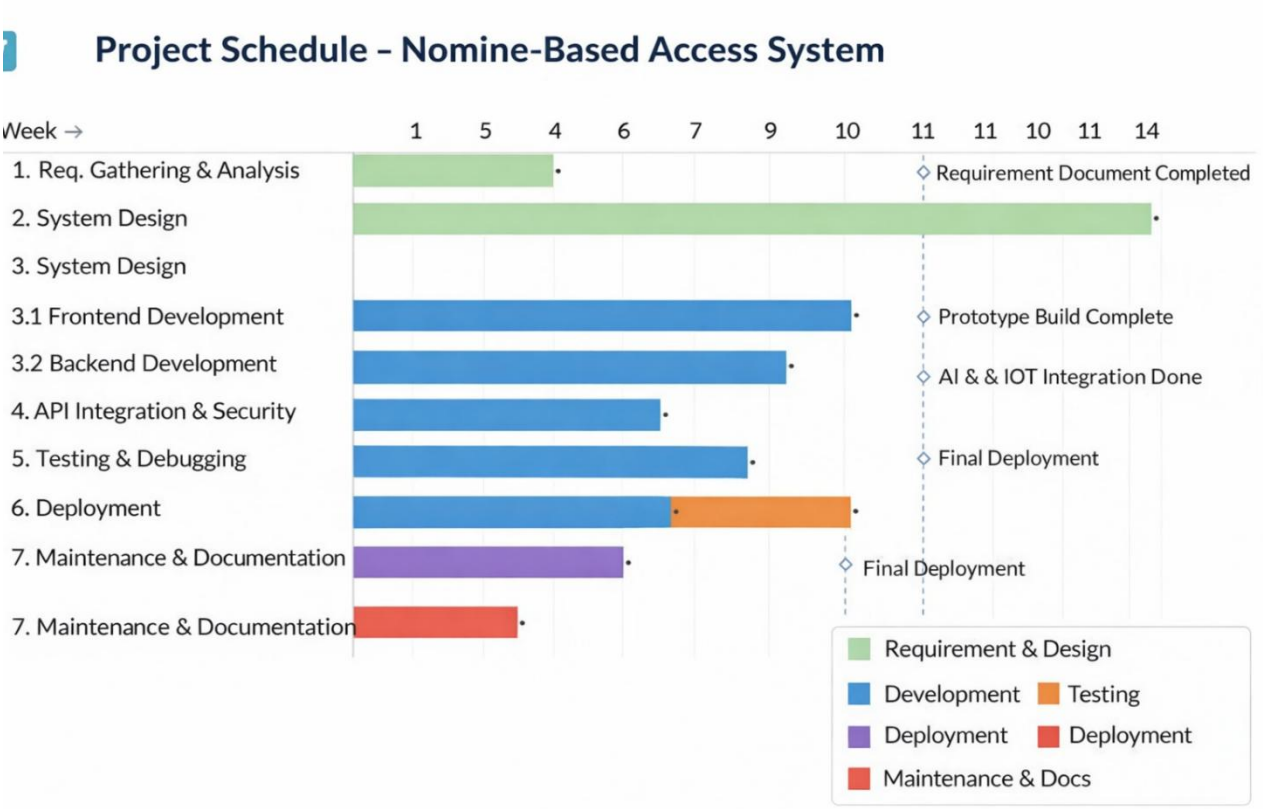
5. **Usability:** The React and Tailwind-based interface must be user-friendly and responsive.

3.3 Planning and Scheduling

Planning and scheduling ensure systematic and time-bound project execution. The project follows the Software Development Life Cycle (SDLC) model.
The development process includes:

- Requirement Analysis
- System Design
- Frontend Development (React + Tailwind)
- Backend Development (Java + Spring Boot)
- Database Integration (MongoDB)
- Authentication Integration (Clerk + JWT)
- API Testing (Postman)
- Deployment

An incremental development approach is followed, where each module (authentication, document management, nominee access) is developed and tested separately before integration.



3.4 Software and Hardware Requirement

Every successful software project depends on choosing the right tools, platforms, and configurations that ensure performance, scalability, and reliability. The **Nominee-Based Access System**, which demand a strong software and hardware foundation. This section describes the detailed software and hardware requirements essential for developing, testing, and deploying the system.

A. Software Requirements

Software resources form the backbone of the project, providing an environment for coding, testing, integration, and deployment. The chosen tools and frameworks focus on **security, efficiency, and cross-platform compatibility.**

1. **Frontend:** React.js with Tailwind CSS Used for building responsive and interactive user interfaces. Enables document management dashboards and nominee access modules.

2. **Backend:** Java with Spring Boot Used for building REST APIs, implementing security configurations, and managing business logic.

3. **Authentication:** Clerk with JWT Clerk manages user identity and session handling, while JWT ensures secure API authorization.

4. **Database:** MongoDB Stores user data, nominee details, and document metadata securely.

5. **Development Tools:**
   - IntelliJ IDEA (Backend Development)
   - VS Code (Frontend Development)
   - Postman (API Testing)
   - Ngrok (Webhook Testing)
   - Git & GitHub (Version Control)
   - Terminal (Project Execution & Debugging)

B. Hardware Requirements

The hardware setup supports software execution, testing, and IoT-based security integration. Since the project involves biometric devices and sensors, a reliable computing environment and embedded systems are essential.

1. Processor

- **Minimum:** Intel Core i5
- **Recommended:** Intel Core i7 or AMD Ryzen 5 A multi-core processor ensures smooth execution of multiple background services such as encryption, AI processing, and database queries simultaneously.

2. RAM (Memory)

- **Minimum Requirement:** 8 GB
- **Recommended:** 16 GB Higher RAM supports concurrent application execution and prevents lag during AI training or when processing large encrypted files.

3. Storage

- **Minimum:** 500 GB HDD
- **Recommended:** 256 GB or higher SSD SSDs offer faster read/write speeds, improving the performance of local databases and AI model loading times.

4. Network Requirements

A **stable high-speed Internet connection** (minimum 5 Mbps) is required for cloud synchronization, and communication between the Web app and backend server.

3.5 Preliminary Product Description

The **Nominee-Based Access System** is a secure digital vault that allows users to store and manage important documents through a **web-based platform**. Users can register, upload documents, and assign trusted nominees with predefined access permissions.

Authentication is handled using **Clerk**, and secure communication between frontend and backend is ensured through **JWT** validation. The backend, developed in Java using Spring Boot, manages business logic and database operations. **MongoDB** securely stores user and document metadata.

The system emphasizes controlled access, structured authorization, and strong cybersecurity principles, ensuring that confidential data remains protected while allowing flexible and secure nominee-based access management.

3.6 Conceptual models

The Nominee-Based Access System follows the **Spiral Model** of software development. The Spiral Model is a risk-driven and iterative development approach that combines elements of both design and prototyping in stages. It is particularly suitable for projects that involve security, authentication mechanisms, and scalable backend architecture, where continuous evaluation and refinement are essential.
In this project, development progresses through multiple cycles (spirals), and each cycle consists of four main phases:

1. **Requirement Analysis and Planning**
2. **Risk Analysis and Design**
3. **Development and Implementation**
4. **Testing and Evaluation**

Each spiral builds upon the previous one, ensuring gradual enhancement and reduction of risks.

Phase 1: Requirement Analysis and Planning
In the first spiral, system requirements were gathered and analyzed. The primary objective was to design a secure digital vault with role-based nominee access control.
Activities performed:

- Identification of user needs
- Defining system objectives
- Selecting technology stack (React, Spring Boot, MongoDB, Clerk, JWT)
- Creating initial documentation

This phase ensured clarity in system scope and functional expectations.

Phase 2: Risk Analysis and System Design
In the second spiral, potential risks related to authentication, token validation, API security, and data protection were analyzed.

Key risk areas:
- Unauthorized API access
- JWT token misuse
- Data breaches
- Improper role-based access control

To reduce these risks:
- Clerk authentication was implemented
- JWT-based authorization was integrated
- Secure API endpoints were configured in Spring Boot
- MongoDB access control was defined

System architecture and database design were finalized in this phase.

Phase 3: Development and Implementation
In the third spiral, the actual development process was carried out.
Development activities included:
- Frontend development using React and Tailwind CSS

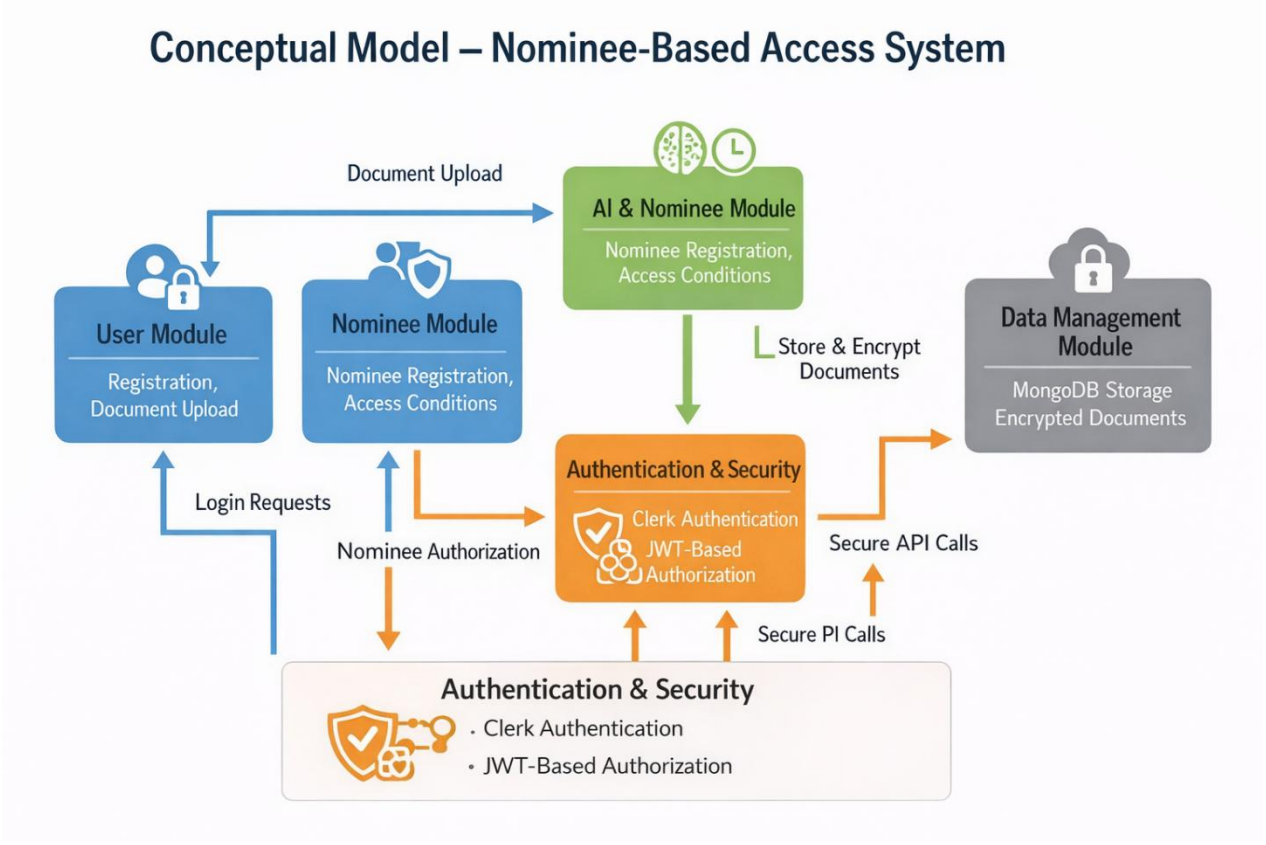
- Backend API development using Java and Spring Boot
- Integration of Clerk authentication
- JWT validation setup in Spring Security
- MongoDB database integration

Each module was developed incrementally and tested independently before integration.

Phase 4: Testing and Evaluation
In the fourth spiral, testing and debugging were performed.

Testing activities included:
- API testing using Postman
- JWT validation testing
- Role-based access verification
- Error handling verification (401, 403 responses)
- Frontend-backend integration testing

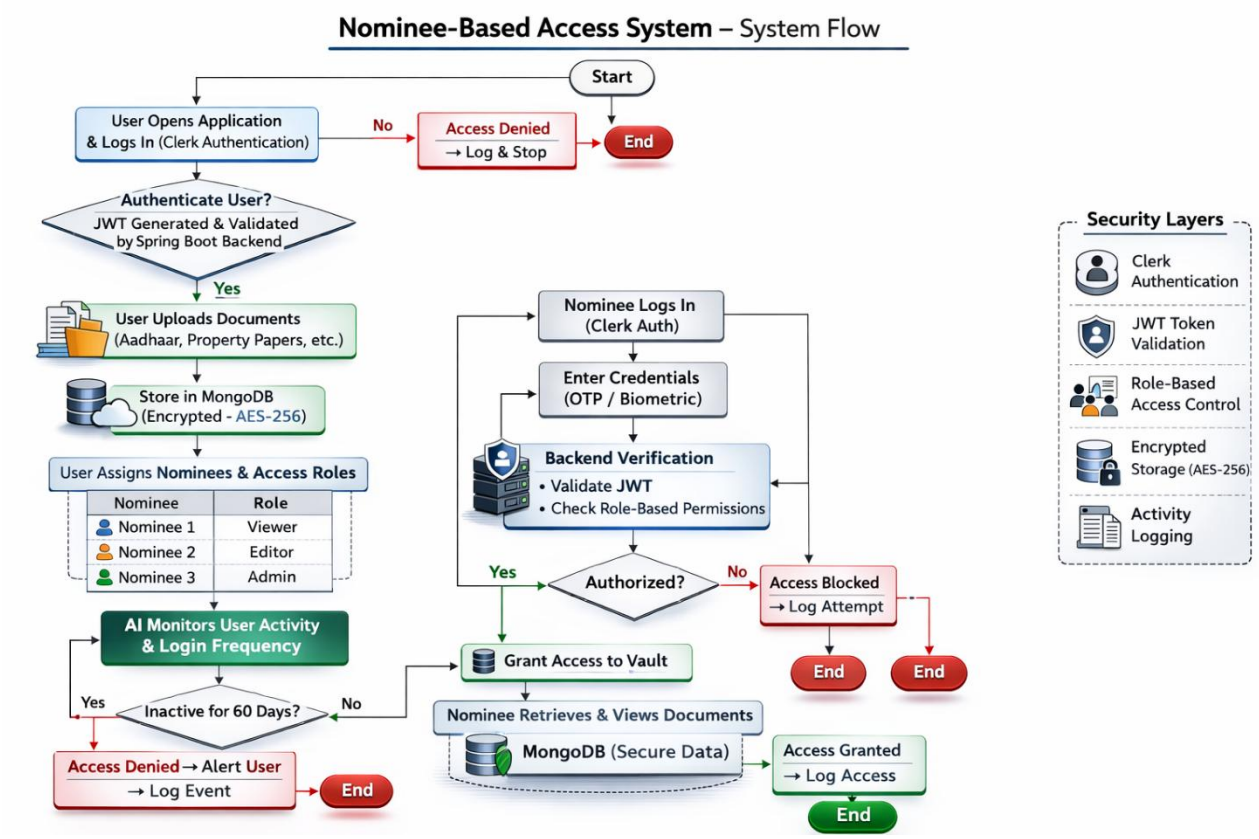
Ngrok was used during development to test webhook and authentication flows securely.
Feedback from testing was used to refine security configurations and improve system stability.



3.6.1 System Flowchart

• User opens the application and logs in using Clerk authentication.
• JWT token is generated and validated by the Spring Boot backend.
• User uploads documents which are stored securely in MongoDB.
• User assigns nominees with predefined access roles.
• Nominee logs in and attempts access.

• Backend verifies JWT and role-based permissions.
• If authorized, access is granted; otherwise, access is denied and logged.



3.6.2 Use Case Diagram

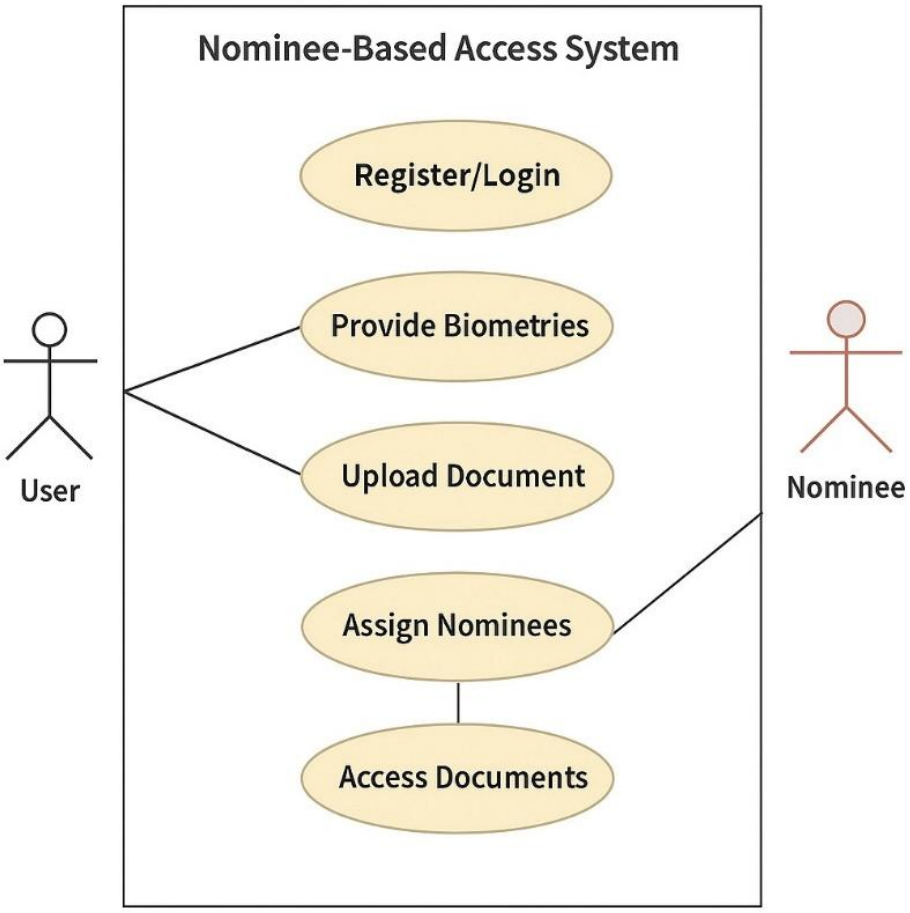
Actors in the system:

- User
- Nominee
- System Admin

Main use cases include:

- User Registration/Login
- Upload Documents
- Assign Nominees
- Role-Based Access Control
- Document Retrieval
- API Authorization

The system ensures that every access request passes through authentication and authorization layers.



3.6.3 Sequence Diagram

• User logs in through Clerk.
• Clerk generates JWT token.
• Frontend sends API request with JWT.
• Spring Boot validates token.
• Backend processes request and interacts with MongoDB.
• Response is sent back to frontend.
• Nominee access is verified through role-based authorization before document retrieval.


User
AuthenticationManager
Database
Nominee
Regiister uer details, biometrrics)
Validate credentials & blometrics
Store user info securely
Confirmation saved
Login (email, password, biometrics)
Verify login credentials
Validate user data
Verified
Access granted
Dashboard displaayed
Add Nominee (nominee info)
Save nominee details
Nominee added successfully
Nominee assigned
Request document access
Fetch and decrypt documents
Return documents
User
AuthenticationManager
Database
Nominee

Chapter 4
System Design

System Design defines the architecture, modules, data flow, and operational logic of the entire system. It acts as a blueprint that converts user requirements into an implementable technical solution. For the Nominee-Based Access System, the design focuses on creating a secure, scalable, and role-based digital vault platform using modern web technologies such as React, Spring Boot, Clerk authentication, JWT authorization, and MongoDB. The system architecture follows a modular structure to ensure maintainability, scalability, and security.

4.1 Basic Modules

1. User Management Module

This module handles:

- User registration and login (via Clerk)
- Profile management
- Secure session handling
- Document upload and management

Authentication is managed through Clerk, and JWT tokens are issued for secure backend communication. Only authenticated users can access their vault.

2. Nominee Management Module

This module allows users to:

- Add trusted nominees
- Define access roles (View-only / Limited access)
- Update or remove nominees

Access to documents is granted only after JWT validation and role verification through the backend. No automatic or inactivity-based access is implemented.

3. Authentication & Authorization Module

This module ensures secure communication and access control.
Security mechanisms include:

- Clerk-based authentication
- JWT-based API authorization
- Role-Based Access Control (RBAC)
- Protected Spring Boot endpoints

Every API request must contain a valid JWT token. Unauthorized requests return HTTP 401 or 403 responses.

4. Data Storage & Encryption Module

This module manages secure document storage.

- MongoDB stores user data, nominee details, and metadata.
- Sensitive information is encrypted.
- HTTPS ensures secure data transmission.

4.2 Data Design

4.2.1 Schema Design

1. User Collection
Primary Key: userId
Attributes:

- userId
- name
- email
- clerkId
- createdAt

Description:
Stores user identity details and authentication reference (linked with Clerk).

2. Nominee Collection
Primary Key: nomineeId
Foreign Key: userId
Attributes:

- nomineeId
- userId
- name
- email
- accessLevel (VIEW / LIMITED)
- createdAt

Description:
Stores nominee details and their assigned access level.

3. Document Collection
Primary Key: documentId
Foreign Key: userId
Attributes:

- documentId
- userId
- fileName
- fileType
- encryptedFilePath
- uploadDate

Description:
Stores document metadata and encrypted file reference.

4. AccessLog Collection
Primary Key: logId
Foreign Keys: userId, nomineeId

Attributes:

- logId
- userId
- nomineeId
- actionType
- timestamp
- status (SUCCESS / DENIED)

Description:

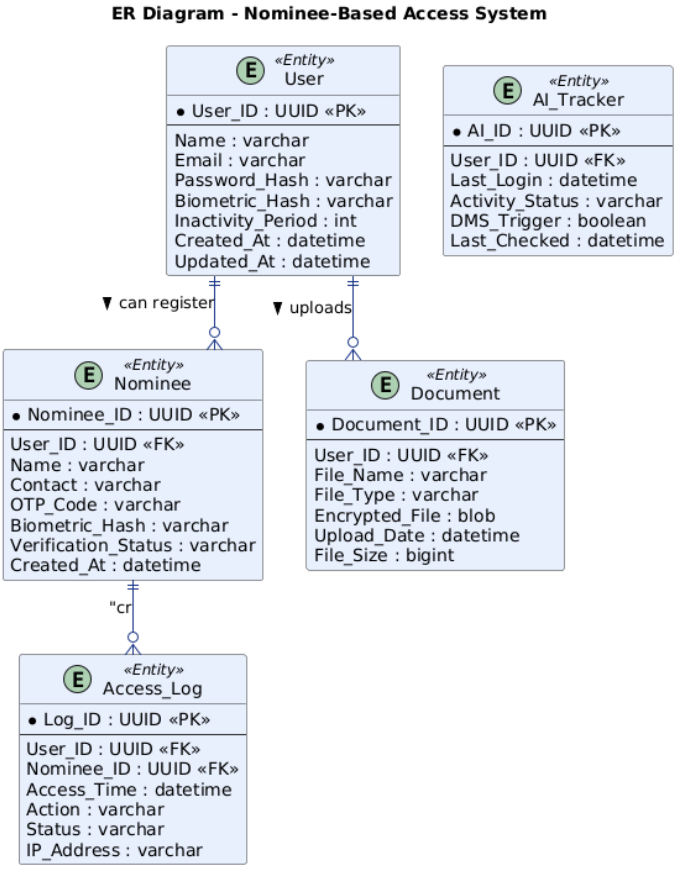
Maintains audit logs of all access attempts.

Entity Relationships

- One User → Many Documents
- One User → Many Nominees
- One Nominee → Many Access Logs
- One User → Many Access Logs

Entity Relationships

- One **User** → can have multiple **Nominees**.
- One **User** → can upload multiple **Documents**.
- One **Nominee** → can have multiple **Access Logs**.
- One **User** → is linked to one **AI Tracker** entry.



4.2.2 Data Integrity and Constraints

1. Entity Integrity
- Every collection has a unique primary identifier.
- No null primary keys allowed.

2. Referential Integrity
- Nominee must reference an existing user.
- Document must reference a valid user.

3. Domain Integrity
- Email must follow valid format.
- AccessLevel must be either VIEW or LIMITED.
- Status must be SUCCESS or DENIED.

4. Access Control Integrity
- Only authenticated users can upload/manage documents.
- Nominees can access only authorized documents.
- All access attempts are logged.

5. Encryption Integrity
- Documents stored in encrypted format.
- Passwords managed securely via Clerk.
- HTTPS used for secure communication.

4.3 Procedural Design

4.3.1 Logic Diagrams

1. Application Start: The process begins when the user opens the Nominee-Based Access System through a web browser. The frontend application built using React initializes and prepares authentication services for user verification.

2. User Login via Clerk: The user enters login credentials on the authentication page. Clerk securely verifies the identity of the user and manages session handling to ensure secure access.

3. JWT Token Generation: After successful authentication, Clerk generates a JSON Web Token (JWT). This token is used to authorize secure communication between the frontend and backend.

4. Secure API Request: The frontend sends API requests to the Spring Boot backend along with the JWT token in the authorization header. This ensures that only authenticated requests are processed.

5. JWT Validation in Backend: The Spring Boot backend validates the received JWT token using security configurations. If the token is valid, the request proceeds; otherwise, access is denied with appropriate HTTP error responses.

6. Dashboard Access: Upon successful validation, the user is redirected to the dashboard. The dashboard displays uploaded documents, nominee details, and available actions.

7. Document Upload Process: When a user uploads a document, the system first validates the file type and size. The document is then encrypted before being stored securely, and its metadata is saved in MongoDB.

8. Nominee Assignment: The user can add a nominee and define access permissions such as view-only or limited access. These nominee details are stored securely and linked to the corresponding user account.

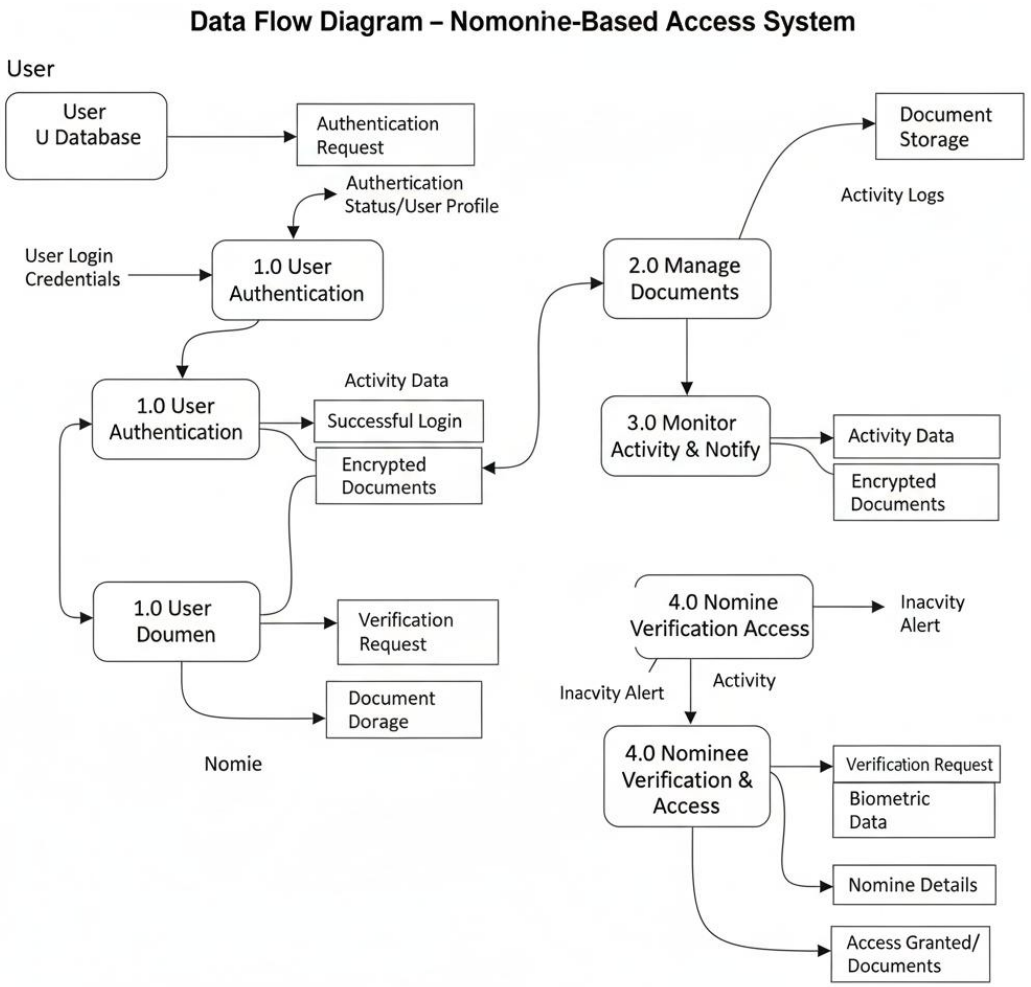
9. Nominee Login: The nominee logs into the system using authenticated credentials. The system verifies identity through Clerk and issues a JWT for secure access.

10. Role-Based Access Verification: When the nominee attempts to access documents, the backend checks the assigned access level. Only documents permitted under the defined role are made accessible.

11. Access Decision: If authorization rules are satisfied, the system grants access to the requested document. If the nominee lacks permission, access is denied with a proper response message.

12. Access Logging: Every access attempt, whether successful or denied, is recorded in the AccessLog collection. This ensures accountability, transparency, and audit tracking.

13. Process End: After completing the requested action, the session remains active until logout. The system continues to enforce authentication and authorization rules for all future requests.



4.3.2 Data Structures

The selection of data structures plays a vital role in ensuring the **speed, security, and reliability** of the **Nominee-Based Access System**. Since the project involves managing users, nominees, encrypted documents, and AI-driven monitoring, efficient data structures are necessary for handling multiple operations simultaneously. The following data structures are used in the system:

JSON Objects
- Used for storing MongoDB documents.
- Used for API request/response payload.

Hash Map
- Used in backend for JWT validation and role mapping.

List (ArrayList)
- Used to store user documents and nominee lists.

Queue (Optional Logging Processing)
- Used for processing access logs in order.

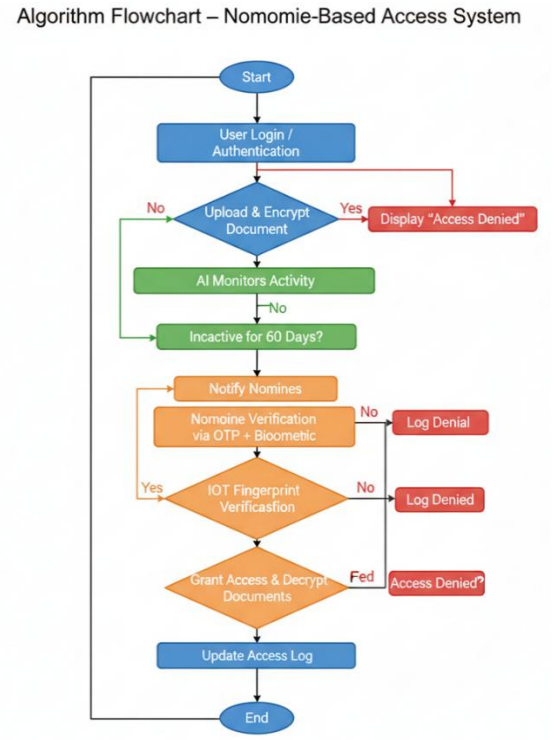
4.3.3. Algorithm Design

1. User Authentication Algorithm
- User logs in via Clerk.
- Clerk verifies credentials.
- JWT generated.
- Backend validates JWT.
- Access granted or denied.

2. Document Upload Algorithm
- Validate file type.
- Encrypt file.
- Store file reference in MongoDB.
- Log upload activity.

3. Nominee Authorization Algorithm
- Verify nominee login.
- Check accessLevel.
- Validate document ownership.
- Grant or deny access.
- Record action in AccessLog.

4. Access Logging Algorithm
- Capture userId / nomineeId.
- Record timestamp.
- Save status.
- Store in AccessLog collection.



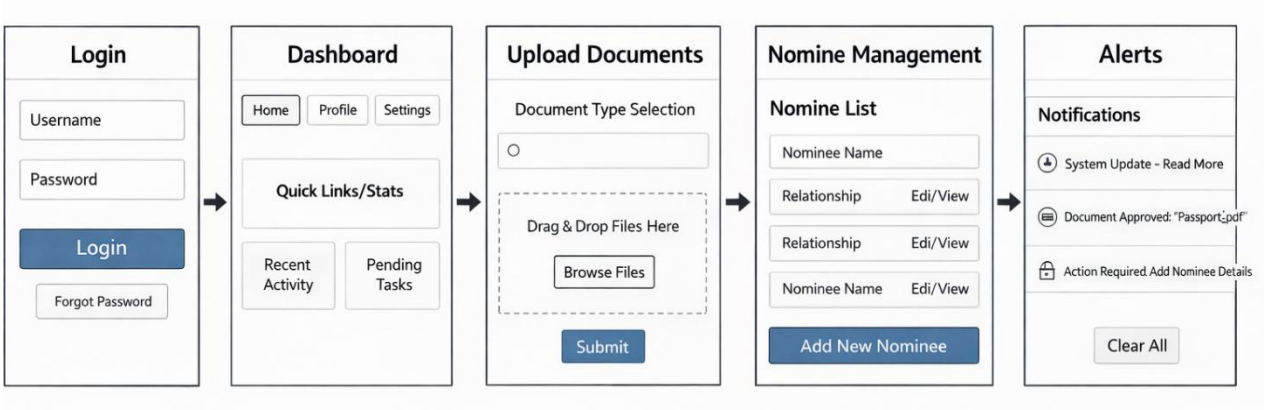
4.4 User Interface Design

The system's **UI/UX design** focuses on simplicity, security, and accessibility. It follows a modern, minimal layout built with **React Native** for mobile compatibility. The UI is developed using React and Tailwind CSS.

Main Screens:

1. Login & Registration Page (Clerk integration)
2. User Dashboard (Document overview)
3. Upload Page (Secure document upload)
4. Nominee Management Page
5. Access Log Page

The UI is responsive, clean, and secure.



4.5 Security Issues

Security is implemented using:

- Clerk authentication
- JWT-based authorization
- Role-Based Access Control
- Encrypted document storage
- HTTPS communication
- Secure Spring Boot API configuration
- Access logging and monitoring

The system avoids IoT and AI-triggered mechanisms and focuses on structured, secure access control.

4.6 Test Cases Design

Testing is conducted to ensure that the system satisfies all **functional** and **non-functional requirements**, performing reliably and securely under various conditions. Different testing techniques such as **unit testing**, **integration testing**, and **security validation** are used to verify the correct functioning of each module in the **Nominee-Based Access System**.

Test Case ID	Description	Input	Expected Output	Result
TC01	**User Login**	**Valid credentials**	**Login successful**	**Pass**
TC02	**Invalid Login**	**Wrong password**	**Access denied**	**Pass**
TC03	**Document Upload**	**Valid file**	**Upload successful**	**Pass**
TC04	**JWT Validation**	**Invalid token**	**401 Unauthorized**	**Pass**
TC05	**Role-Based Access**	**Unauthorized nominee**	**Access denied**	**Pass**
TC06	**Authorized Access**	**Valid nominee**	**Access granted**	**Pass**
TC07	**Document Encryption**	**Upload sensitive file**	**Stored encrypted**	**Pass**

Chapter 5
Implementation And Testing

5.1 implementation Approaches

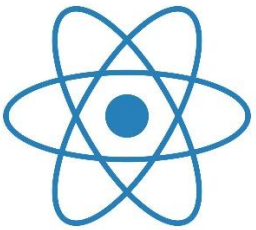
The implementation of the Nominee-Based Access System followed a structured full-stack development approach using modern web technologies. The system was developed in a modular manner, separating the frontend, backend, authentication, and database layers to ensure scalability and maintainability. The frontend was built using React.js with Tailwind CSS to create a responsive and user-friendly interface. Clerk was integrated for secure user authentication and session management. After successful login, Clerk generates a JWT token which is used for secure communication between the frontend and backend. Protected routes were implemented to restrict access to authorized users only. API calls were handled using secure HTTP requests, ensuring that every request carried proper authorization credentials.

On the backend side, the system was implemented using Java and Spring Boot to develop RESTful APIs for user management, nominee handling, and document operations. Spring Security was configured to validate JWT tokens and enforce role-based access control (RBAC). MongoDB was used as the database to store user profiles, nominee details, document metadata, and access logs. The implementation ensured that sensitive information was securely stored and transmitted over HTTPS. Logging mechanisms were added to track all access attempts for audit and monitoring purposes. The development process followed an incremental approach, where each module—authentication, document management, and nominee access—was implemented and tested individually before final integration. This approach ensured reliability, security, and proper validation of system functionalities.

1. **React JS**

   React.js is used as the primary frontend library for developing the user interface of the Nominee-Based Access System. It follows a component-based architecture, which allows the application to be divided into reusable and manageable UI components such as login forms, dashboards, document upload modules, and nominee management panels. React improves application performance through the use of the Virtual DOM, which updates only the necessary parts of the interface instead of reloading the entire page.

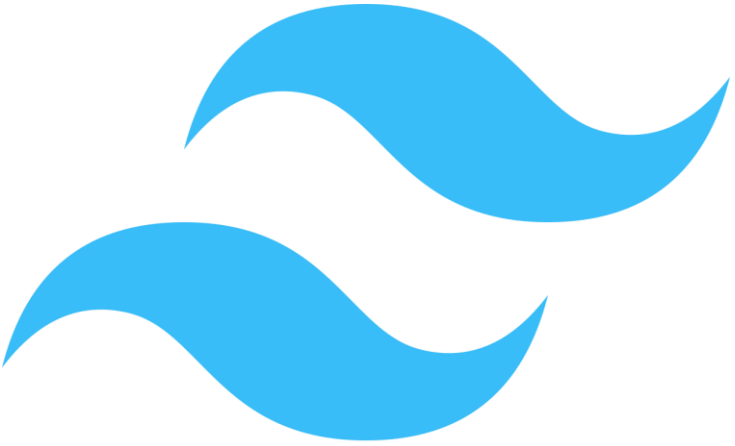
   In this project, React handles routing, state management, and API communication with the backend. It ensures that the user interface remains dynamic and responsive. Protected routes are implemented to restrict unauthorized access to certain pages. React works seamlessly with Clerk authentication and integrates easily with backend APIs developed in Spring Boot.



2. Tailwind CSS

Tailwind CSS is a utility-first CSS framework used to design the frontend interface. It allows rapid UI development by applying pre-defined utility classes directly in HTML/JSX. This approach reduces the need for writing custom CSS files and ensures consistency across the application.

In the Nominee-Based Access System, Tailwind CSS is used to create a clean, modern, and responsive layout. It supports mobile-first design, ensuring compatibility across different screen sizes. The framework improves development speed while maintaining a professional and structured user interface.



3. Java

Java is used as the backend programming language due to its robustness, security, and enterprise-level capabilities. It provides strong object-oriented programming features and supports scalable application development. Java is widely used in secure web applications because of its reliability and performance.

In this project, Java handles business logic, nominee access validation, document processing, and server-side security enforcement. It ensures that all operations related to user data and access control are executed securely and efficiently.

4. **Spring Boot**

Spring Boot is used as the backend framework to develop RESTful APIs and manage application logic. It simplifies backend configuration and provides built-in support for dependency injection, security configuration, and database integration.

In the Nominee-Based Access System, Spring Boot is responsible for handling API requests, validating JWT tokens, implementing role-based access control, and connecting with MongoDB. Spring Security is configured within the framework to protect API endpoints from unauthorized access.

5. **MongoDB**

MongoDB is a NoSQL document-oriented database used to store user data, nominee details, document metadata, and access logs. Unlike traditional relational databases, MongoDB stores data in JSON-like format, which provides flexibility in handling structured and semi-structured data.

In this project, MongoDB ensures scalable and efficient data storage. It integrates smoothly with Spring Boot using Spring Data MongoDB repositories. The database stores references to encrypted files and maintains structured relationships between users and nominees.

6. Clerk

Clerk is used for secure authentication and identity management. It handles user registration, login, session management, and multi-factor authentication. Clerk simplifies the implementation of secure authentication flows without requiring complex custom coding.

In this system, Clerk generates JWT tokens after successful login. These tokens are used by the frontend to access protected backend APIs. Clerk ensures that only verified users can interact with the digital vault.

7. JWT (JSON Web Token)

JWT (JSON Web Token) is used for secure and stateless authentication between frontend and backend. After successful authentication through Clerk, a JWT token is issued to the user and included in API request headers.

Spring Boot validates the JWT token before granting access to any protected endpoint. This ensures secure communication and prevents unauthorized API access. JWT-based authentication enhances security while maintaining system performance.


JWT
JSON WEB Token

5.2 Coding Details and Efficiency

5.2.1 Coding Details

The Nominee-Based Access System was developed using a modular and layered coding structure to ensure maintainability and scalability. The frontend was implemented using React.js, where reusable components were created for login, dashboard, document upload, and nominee management. Each component follows a structured folder hierarchy, separating UI logic, API calls, and state management. Tailwind CSS utility classes were used directly within JSX files to maintain consistent styling without creating large external CSS files.

Authentication was integrated using Clerk, and JWT tokens were verified on every protected API request. Input validation was implemented at both frontend and backend levels to prevent invalid data submission. Logging functionality was added to track access attempts and system activities, ensuring proper audit trails.

Source Code:

(Frontend)
Main.jsx:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import {ClerkProvider} from "@clerk/clerk-react";

const clerkPubKey = import.meta.env.VITE_CLERK_PUBLISHABLE_KEY;

createRoot(document.getElementById('root')).render(
  <ClerkProvider publishableKey={clerkPubKey}>
    <App />
  </ClerkProvider>
)
```

App.jsx:

```
import {BrowserRouter, Route, Routes} from "react-router-dom";
import Landing from "./pages/Landing.jsx";
import Dashboard from "./pages/Dashboard.jsx";
import Upload from "./pages/Upload.jsx";
import MyFiles from "./pages/MyFiles.jsx";
import Subscription from "./pages/Subscription.jsx";
import Transactions from "./pages/Transactions.jsx";
import {RedirectToSignIn, SignedIn, SignedOut} from "@clerk/clerk-react";
import {Toaster} from "react-hot-toast";
import {UserCreditsProvider} from "./context/UserCreditsContext.jsx";
import PublicFileView from "./pages/PublicFileView.jsx";

const App = () => {
   return (
      <UserCreditsProvider>
         <BrowserRouter>
```

```
            <Toaster />
            <Routes>
              <Route path="/" element={<Landing />} />
              <Route path="/dashboard" element={
                <>
                  <SignedIn><Dashboard /></SignedIn>
                  <SignedOut><RedirectToSignIn /></SignedOut>
                </>
              } />
              <Route path="/upload" element={
                <>
                  <SignedIn><Upload /></SignedIn>
                  <SignedOut><RedirectToSignIn /></SignedOut>
                </>
              } />
              <Route path="/my-files" element={
                <>
                  <SignedIn><MyFiles /></SignedIn>
                  <SignedOut><RedirectToSignIn /></SignedOut>
                </>
              } />
              <Route path="/subscriptions" element={
                <>
                  <SignedIn><Subscription /></SignedIn>
                  <SignedOut><RedirectToSignIn /></SignedOut>
                </>
              } />
              <Route path="/transactions" element={
                <>
                  <SignedIn><Transactions /></SignedIn>
                  <SignedOut><RedirectToSignIn /></SignedOut>
                </>
              } />
              <Route path="file/:fileId" element={
                <>
                  <PublicFileView />
                </>
              }/>
              <Route path="/*" element={<RedirectToSignIn />} />
            </Routes>
          </BrowserRouter>
        </UserCreditsProvider>
    )
}
export default App;
```

Dashboard.jsx:

```
import DashboardLayout from "../layout/DashboardLayout.jsx";
import {useAuth} from "@clerk/clerk-react";
import {useContext, useEffect, useState} from "react";
import {UserCreditsContext} from "../context/UserCreditsContext.jsx";
```

```
import axios from "axios";
import {apiEndpoints} from "../util/apiEndpoints.js";
import {Loader2} from "lucide-react";
import DashboardUpload from "../components/DashboardUpload.jsx";
import RecentFiles from "../components/RecentFiles.jsx";

const Dashboard = () => {
    const [files, setFiles] = useState([]);
    const [uploadFiles, setUploadFiles] = useState([]);
    const [uploading, setUploading] = useState(false);
    const [loading, setLoading] = useState(false);
    const [message, setMessage] = useState('');
    const [messageType, setMessageType] = useState('');
    const [remainingUploads, setRemainingUploads] = useState(5);
    const {getToken} = useAuth();
    const { fetchUserCredits } = useContext(UserCreditsContext);
    const MAX_FILES = 5;

    useEffect(() => {
        const fetchRecentFiles = async () => {
            setLoading(true);
            try {
                const token = await getToken();
                console.log("User Token:", token);
                const res = await axios.get(apiEndpoints.FETCH_FILES, {
                    headers: {
                        'Authorization': `Bearer ${token}`,
                    }
                });

                // Sort by uploadedAt and take only the 5 most recent files
                const sortedFiles = res.data.sort((a, b) =>
                    new Date(b.uploadedAt) - new Date(a.uploadedAt)
                ).slice(0, 5);
                setFiles(sortedFiles);
            } catch (error) {
                console.error("Error fetching recent files:", error);
            } finally {
                setLoading(false);
            }
        };
        fetchRecentFiles();
    }, [getToken]);

    const handleFileChange = (e) => {
        const selectedFiles = Array.from(e.target.files);

        // Check if adding these files would exceed the limit
        if (uploadFiles.length + selectedFiles.length > MAX_FILES) {
            setMessage(`You can only upload a maximum of ${MAX_FILES} files at once.`);
            setMessageType('error');
```

```
        return;
    }

    // Add the new files to the existing files
    setUploadFiles(prevFiles => [...prevFiles, ...selectedFiles]);
    setMessage('');
    setMessageType('');
};

// Remove a file from the upload list
const handleRemoveFile = (index) => {
    setUploadFiles(prevFiles => prevFiles.filter((_, i) => i !== index));
    setMessage('');
    setMessageType('');
};

// Calculate remaining uploads
useEffect(() => {
    setRemainingUploads(MAX_FILES - uploadFiles.length);
}, [uploadFiles]);

// Handle file upload
const handleUpload = async () => {
    if (uploadFiles.length === 0) {
        setMessage('Please select at least one file to upload.');
        setMessageType('error');
        return;
    }

    if (uploadFiles.length > MAX_FILES) {
        setMessage(`You can only upload a maximum of ${MAX_FILES} files at once.`);
        setMessageType('error');
        return;
    }

    setUploading(true);
    setMessage('Uploading files...');
    setMessageType('info');

    const formData = new FormData();
    uploadFiles.forEach(file => formData.append('files', file));

    try {
        const token = await getToken();
        const response = await axios.post(apiEndpoints.UPLOAD_FILE, formData, {
            headers: {
                'Authorization': `Bearer ${token}`,
                'Content-Type': 'multipart/form-data'
            }
        });
```

```
        setMessage('Files uploaded successfully!');
        setMessageType('success');
        setUploadFiles([]);

        // Refresh the recent files list
        const res = await axios.get(apiEndpoints.FETCH_FILES, {
          headers: {
            'Authorization': `Bearer ${token}`,
          }
        });

        // Sort by uploadedAt and take only the 5 most recent files
        const sortedFiles = res.data.sort((a, b) =>
          new Date(b.uploadedAt) - new Date(a.uploadedAt)
        ).slice(0, 5);

        setFiles(sortedFiles);

        // Refresh user credits immediately after successful upload
        await fetchUserCredits();
      } catch (error) {
        console.error('Error uploading files:', error);
        setMessage(error.response?.data?.message || 'Error uploading files. Please try again.');
        setMessageType('error');
      } finally {
        setUploading(false);
      }
    };

    return (
      <DashboardLayout activeMenu="Dashboard">
        <div className="p-6">
          <h1 className="text-2xl font-bold mb-6">My Drive</h1>
          <p className="text-gray-600 mb-6">Upload, manage, and share your files
securely</p>
          {message && (
            <div className={`mb-6 p-4 rounded-lg flex items-center gap-3 ${
              messageType === 'error' ? 'bg-red-50 text-red-700' :
                messageType === 'success' ? 'bg-green-50 text-green-700' :
                  'bg-purple-50 text-purple-700'
            }`}>
              {message}
            </div>
          )}
          <div className="flex flex-col md:flex-row gap-6">
            {/*Left column*/}
            <div className="w-full md:w-[40%]">
              <DashboardUpload
                files={uploadFiles}
                onFileChange={handleFileChange}
                onUpload={handleUpload}
```

```
                    uploading={uploading}
                    onRemoveFile={handleRemoveFile}
                    remainingUploads={remainingUploads}
                  />
                </div>

                {/*right column*/}
                <div className="w-full md:w-[60%]">
                  {loading ? (
                    <div className="bg-white rounded-lg shadow p-8 flex flex-col items-center
justify-center min-h-[300px]">
                      <Loader2 size={40} className="text-purple-500 animate-spin mb-4" />
                      <p className="text-gray-500">Loading your files...</p>
                    </div>
                  ) : (
                    <RecentFiles files={files} />
                  )}
                </div>
              </div>
            </div>
        </DashboardLayout>
    )
}

export default Dashboard;
```

Landing.jsx:

```
import HeroSection from '../components/landing/HeroSection';
import FeaturesSection from '../components/landing/FeaturesSection';
import PricingSection from '../components/landing/PricingSection';
import TestimonialsSection from '../components/landing/TestimonialsSection';
import CTASection from '../components/landing/CTASection';
import Footer from '../components/landing/Footer';
import {features, pricingPlans, testimonials} from "../assets/data.js";
import {useClerk, useUser} from '@clerk/clerk-react';
import {useEffect} from "react";
import {useNavigate} from "react-router-dom";
import { Fence } from 'lucide-react';

const Landing = () => {
    const {openSignIn, openSignUp} = useClerk();
    const {isSignedIn} = useUser();
    const navigate = useNavigate();

    useEffect(() => {
        if (isSignedIn) {
            navigate("/dashboard");
        }
    }, [isSignedIn, navigate]);
    return (
        <div className="landing-page bg-gradient-to-b from-gray-50 to-gray-100">
```

```
        {/* Hero Section*/}
        <HeroSection openSignIn={openSignIn} openSignUp={openSignUp}/>
        {/* Features section*/}
        <FeaturesSection features={features}/>

        {/* Pricing section*/}
        <PricingSection pricingPlans={pricingPlans} openSignUp={openSignUp}/>

        {/* Testimonials section*/}
        <TestimonialsSection testimonials={testimonials}/>

        {/* CTA section*/}
        <CTASection openSignUp={openSignUp}/>

        {/* Footer section*/}
        <Footer />
      </div>
    )
}

export default Landing;
```

MyFiles.jsx:

```
import DashboardLayout from "../layout/DashboardLayout.jsx";
import {useEffect, useState} from "react";
import { File, FileIcon, FileText, Grid, Image, List, Music, Video } from "lucide-react";
import {useAuth} from "@clerk/clerk-react";
import axios from "axios";
import toast from "react-hot-toast";
import {useNavigate} from "react-router-dom";
import FileCard from "../components/FileCard.jsx";
import {apiEndpoints} from "../util/apiEndpoints.js";
import ConfirmationDialog from "../components/ConfirmationDialog.jsx";
import LinkShareModal from "../components/LinkShareModal.jsx";
import FileListRow from "../components/FileListRow.jsx";

const MyFiles = () => {
    const [files, setFiles] = useState([]);
    const [viewMode, setViewMode] = useState("list");
    const {getToken} = useAuth();
    const navigate = useNavigate();
    const [deleteConfirmation, setDeleteConfirmation] = useState({
        isOpen: false,
        fileId: null
    });
    const [shareModal, setShareModal] = useState({
        isOpen: false,
        fileId: null,
        link: ""
    });
```

```
  //fetching the files for a logged in user
  const fetchFiles = async () => {
    try {
      const token = await getToken();
      console.log(token);
      const response = await axios.get(apiEndpoints.FETCH_FILES, {headers: {Authorization:
`Bearer ${token}`}});
      if (response.status === 200) {
        setFiles(response.data);
      }
    }catch (error) {
      console.error('Error fetching the files from server: ', error);
      toast.error('Error fetching the files from server: ', error.message);
    }
  }

  //Toggles the public/private status of a file
  const togglePublic = async (fileToUpdate) => {
    try {
      const token = await getToken();
      await axios.patch(apiEndpoints.TOGGLE_FILE(fileToUpdate.id), {}, {headers:
{Authorization: `Bearer ${token}`}});
      console.log('data', fileToUpdate);
      setFiles(files.map((file) => file.id === fileToUpdate.id ? {...file, isPublic: !file.isPublic}:
file));
    }catch (error) {
      console.error('Error toggling file status', error);
      toast.error('Error toggling file status: ', error.message);
    }
  }

  //Handle file download
  const handleDownload = async (file) => {
    try {
      const token = await getToken();
      const response = await axios.get(apiEndpoints.DOWNLOAD_FILE(file.id), {headers:
{Authorization: `Bearer ${token}`}, responseType: 'blob'});

      // create a blob url and trigger download
      const url = window.URL.createObjectURL(new Blob([response.data]));
      const link = document.createElement("a");
      link.href = url;
      link.setAttribute("download", file.name);
      document.body.appendChild(link);
      link.click();
      link.remove();
      window.URL.revokeObjectURL(url); // clean up the object url
    }catch (error) {
      console.error('Download failed', error);
      toast.error('Error downloading file', error.message);
    }
```

```
    }

    //Closes the delete confirmation modal
    const closeDeleteConfirmation = () => {
        setDeleteConfirmation({
            isOpen: false,
            fileId: null
        })
    }

    //Opens the delete confirmation modal
    const openDeleteConfirmation = (fileId) => {
        setDeleteConfirmation({
            isOpen: true,
            fileId
        })
    }

    //opens the share link modal
    const openShareModal = (fileId) => {
        const link = `${window.location.origin}/file/${fileId}`;
        setShareModal({
            isOpen: true,
            fileId,
            link
        });
    }

    //close the share link modal
    const closeShareModal = () => {
        setShareModal({
            isOpen: false,
            fileId: null,
            link: ""
        });
    }

    //Delete a file after confirmation
    const handleDelete = async () => {
        const fileId = deleteConfirmation.fileId;
        if (!fileId) return;

        try {
            const token = await getToken();
            const response = await axios.delete(apiEndpoints.DELETE_FILE(fileId), {headers:
{Authorization: `Bearer ${token}`}});
            if (response.status === 204) {
                setFiles(files.filter((file) => file.id !== fileId));
                closeDeleteConfirmation();
            } else {
                toast.error('Error deleting file');
```

```
        }
      }catch (error) {
        console.error('Error deleting file', error);
        toast.error('Error deleting file', error.message);
      }
    }

    useEffect(() => {
      fetchFiles();
    }, [getToken]);

    const getFileIcon = (file) => {
      const extenstion = file.name.split('.').pop().toLowerCase();

      if (['jpg', 'jpeg', 'png', 'gif', 'svg', 'webp'].includes(extenstion)) {
        return <Image size={24} className="text-purple-500" />
      }

      if (['mp4', 'webm', 'mov', 'avi', 'mkv'].includes(extenstion)) {
        return <Video size={24} className="text-blue-500" />
      }

      if (['mp3', 'wav', 'ogg', 'flac', 'm4a'].includes(extenstion)) {
        return <Music size={24} className="text-green-500" />
      }

      if (['pdf', 'doc', 'docx', 'txt', 'rtf'].includes(extenstion)) {
        return <FileText size={24} className="text-amber-500" />
      }

      return <FileIcon size={24} className="text-purple-500" />
    }

    return (
      <DashboardLayout activeMenu="My Files">
        <div className="p-6">
          <div className="flex justify-between items-center mb-6">
            <h2 className="text-2xl font-bold">My Files {files.length}</h2>
            <div className="flex items-center gap-3">
              <List
                onClick={() => setViewMode("list")}
                size={24}
                className={`cursor-pointer transition-colors ${viewMode === 'list' ? 'text-
blue-600': 'text-gray-400 hover:text-gray-600'}`} />
              <Grid
                size={24}
                onClick={() => setViewMode("grid")}
                className={`cursor-pointer transition-colors ${viewMode === 'grid' ? 'text-
blue-600': 'text-gray-400 hover:text-gray-600'}`} />
            </div>
          </div>
```

```
          {files.length === 0 ? (
            <div className="bg-white rounded-lg shadow p-12 flex flex-col items-center
justify-center">
              <File
                size={60}
                className="text-purple-300 mb-4"
              />
              <h3 className="text-xl font-medium text-gray-700 mb-2">
                No files uploaded yet
              </h3>
              <p className="text-gray-500 text-center max-w-md mb-6">
                Start uploading files to see them listed here. you can upload
                documents, images, and other files to share and manage them securely.
              </p>
              <button
                onClick={() => navigate('/upload')}
                className="px-4 py-2 bg-purple-500 text-white rounded-md hover:bg-purple-
600 transition-colors">
                Go to Upload
              </button>
            </div>
          ): viewMode === "grid" ? (
            <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-
6">
              {files.map((file) => (
                <FileCard
                  key={file.id}
                  file={file}
                  onDelete={openDeleteConfirmation}
                  onTogglePublic={togglePublic}
                  onDownload={handleDownload}
                  onShareLink={openShareModal}
                />
              ))}
            </div>
          ) : (
            <div className="overflow-x-auto bg-white rounded-lg shadow">
              <table className="min-w-full">
                <thead className="bg-gray-50 border-b border-gray-200">
                <tr>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">Name</th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">Size</th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">Uploaded</th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">Sharing</th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">Actions</th>
```

```
                    </tr>
                    </thead>
                    <tbody className="divide-y divide-gray-200">
                      {files.map((file) => (
                        <FileListRow
                          key={file.id}
                          file={file}
                          onDownload={handleDownload}
                          onDelete={openDeleteConfirmation}
                          onTogglePublic={togglePublic}
                          onShareLink={openShareModal}
                          getFileIcon={getFileIcon}
                        />
                      ))}
                    </tbody>
                  </table>
                </div>
              )}
              {/* Delete confiramtion dialog*/}
              <ConfirmationDialog
                isOpen={deleteConfirmation.isOpen}
                onClose={closeDeleteConfirmation}
                title="Delete File"
                message="Are you sure want to delete this file? This action cannot be undone."
                confirmText="Delete"
                cancelText="Cancel"
                onConfirm={handleDelete}
                confirmButtonClass="bg-red-600 hover:bg-red-700"
              />

              {/* Share link modal */}
              <LinkShareModal
                isOpen={shareModal.isOpen}
                onClose={closeShareModal}
                link={shareModal.link}
                title="Share File"
              />
            </div>
          </DashboardLayout>
        )
}

export default MyFiles;
```

PublicFileView.jsx:

```
import {useEffect, useState} from "react";
import {useParams} from "react-router-dom";
import {useAuth} from "@clerk/clerk-react";
import axios from "axios";
import {apiEndpoints} from "../util/apiEndpoints.js";
import toast from "react-hot-toast";
```

```
import {Copy, Download, File, Info, Share2} from "lucide-react";
import LinkShareModal from "../components/LinkShareModal.jsx";

const PublicFileView = () => {
    const [file, setFile] = useState(null);
    const [error, setError] = useState(null);
    const [isLoading, setIsLoading] = useState(false);
    const [shareModal, setShareModal] = useState({
        isOpen: false,
        link: ""
    });
    const {getToken} = useAuth();
    const {fileId} = useParams();

    useEffect(() => {
        const getFile = async () => {
            setIsLoading(true);
            try {
                // Re-added token fetching and authorization header
                const res = await axios.get(
                    apiEndpoints.PUBLIC_FILE_VIEW(fileId)
                );
                setFile(res.data);
                setError(null);
            } catch (err) {
                console.error("Error fetching file:", err);
                setError(
                    "Could not retrieve file. The link may be invalid or the file may have been removed."
                );
            } finally {
                setIsLoading(false);
            }
        };
        getFile();
    }, [fileId, getToken]);

    const handleDownload = async () => {
        try {
            // This endpoint might also require a token depending on your backend setup
            const response = await axios.get(
                apiEndpoints.DOWNLOAD_FILE(fileId),
                {
                    responseType: "blob",
                }
            );

            const url = window.URL.createObjectURL(new Blob([response.data]));
            const link = document.createElement("a");
            link.href = url;
            link.setAttribute("download", file.name); // Use the actual file name
            document.body.appendChild(link);
```

```
      link.click();
      link.remove();
      window.URL.revokeObjectURL(url); // Clean up the object URL
    } catch (err) {
      console.error("Download failed:", err);
      toast.error("Sorry, the file could not be downloaded.");
    }
};

const openShareModal = () => {
    setShareModal({
      isOpen: true,
      link: window.location.href,
    });
};

const closeShareModal = () => {
    setShareModal({
      isOpen: false,
      link: "",
    });
};

if (isLoading) {
    return (
      <div className="flex justify-center items-center h-screen bg-gray-50">
        <p className="text-gray-600">Loading file...</p>
      </div>
    );
}

if (error) {
    return (
      <div className="flex justify-center items-center h-screen bg-gray-50">
        <div className="text-center p-8 bg-white rounded-lg shadow-md">
          <h2 className="text-xl font-semibold text-red-600">Error</h2>
          <p className="text-gray-600 mt-2">{error}</p>
        </div>
      </div>
    );
}

if (!file) return null;

return (
    <div className="bg-gray-50 min-h-screen">
      <header className="p-4 border-b bg-white">
        <div className="container mx-auto flex justify-between items-center">
          <div className="flex items-center gap-2">
            <Share2 className="text-blue-600" />
            <span className="font-bold text-xl text-gray-800">CloudShare</span>
```

```
            </div>
            <button
              onClick={openShareModal}
              className="flex items-center gap-2 px-4 py-2 bg-blue-50 text-blue-600 rounded-
lg hover:bg-blue-100 transition-colors"
            >
              <Copy size={18} />
              Share Link
            </button>
          </div>
        </header>

        {/* Main Content */}
        <main className="container mx-auto p-4 md:p-8 flex justify-center">
          <div className="w-full max-w-3xl">
            <div className="bg-white border border-gray-200 rounded-lg shadow-sm p-8 text-
center">
              <div className="flex justify-center mb-4">
                <div className="w-20 h-20 bg-blue-100 rounded-full flex items-center justify-
center">
                  <File size={40} className="text-blue-500" />
                </div>
              </div>

              <h1 className="text-2xl font-semibold text-gray-800 break-words">
                {file.name}
              </h1>
              <p className="text-sm text-gray-500 mt-2">
                {(file.size / 1024).toFixed(2)} KB
                <span className="mx-2">&bull;</span>
                Shared on {new Date(file.uploadedAt).toLocaleDateString()}
              </p>

              <div className="my-6">
        <span className="inline-block bg-gray-100 text-gray-600 text-xs font-medium px-3 py-
1 rounded-full uppercase">
          {file.type || "File"}
        </span>
              </div>

              <div className="flex justify-center gap-4 my-8">
                <button
                  onClick={handleDownload}
                  className="flex items-center gap-2 px-6 py-3 bg-gray-800 text-white
rounded-lg hover:bg-gray-900 transition-colors shadow"
                >
                  <Download size={18} />
                  Download File
                </button>
              </div>
```

```
            <hr className="my-8" />

            <div>
              <h3 className="text-lg font-semibold text-left text-gray-800 mb-4">
                File Information
              </h3>
              <div className="text-left text-sm space-y-3">
                <div className="flex justify-between">
                  <span className="text-gray-500">File Name:</span>
                  <span className="text-gray-800 font-medium break-all">
          {file.name}
        </span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-500">File Type:</span>
                  <span className="text-gray-800 font-medium">{file.type}</span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-500">File Size:</span>
                  <span className="text-gray-800 font-medium">
          {(file.size / 1024).toFixed(2)} KB
        </span>
                </div>
                <div className="flex justify-between">
                  <span className="text-gray-500">Shared:</span>
                  <span className="text-gray-800 font-medium">
          {new Date(file.uploadedAt).toLocaleDateString()}
        </span>
                </div>
              </div>
            </div>

            <div className="mt-6 bg-blue-50 border border-blue-200 text-blue-800 p-4
rounded-lg flex items-center gap-4">
              <Info size={20} />
              <p className="text-sm">
                This file has been shared publicly. Anyone with this link can view
                and download it.
              </p>
            </div>
          </div>
        </main>
        <LinkShareModal
          isOpen={shareModal.isOpen}
          onClose={closeShareModal}
          link={shareModal.link}
          title="Share File"
        />
      </div>
  )
```

```
}

export default PublicFileView;
```

Subscription.jsx:

```
import DashboardLayout from "../layout/DashboardLayout.jsx";
import {useContext, useEffect, useRef, useState} from "react";
import {useAuth, useUser} from "@clerk/clerk-react";
import {UserCreditsContext} from "../context/UserCreditsContext.jsx";
import axios from "axios";
import {apiEndpoints} from "../util/apiEndpoints.js";
import {AlertCircle, Check, CreditCard, Loader2} from "lucide-react";

const Subscription = () => {
    const [processingPayment, setProcessingPayment] = useState(false);
    const [message, setMessage] = useState("");
    const [messageType, setMessageType] = useState("");
    const [razorpayLoaded, setRazorpayLoaded] = useState(false);

    const {getToken} = useAuth();
    const razorpayScriptRef = useRef(null);
    const {credits, setCredits, fetchUserCredits} = useContext(UserCreditsContext);

    const {user} = useUser();

    // Plans configuration
    const plans = [
        {
            id: "premium",
            name: "Premium",
            credits: 500,
            price: 50,
            features: [
                "Upload up to 500 files",
                "Access to all basic features",
                "Priority support"
            ],
            recommended: false
        },
        {
            id: "ultimate",
            name: "Ultimate",
            credits: 5000,
            price: 250,
            features: [
                "Upload up to 5000 files",
                "Access to all premium features",
                "Priority support",
                "Advanced analytics"
            ],
            recommended: true
```

```
    }
];

// Load Razorpay script
useEffect(() => {
    if (!window.Razorpay) {
        const script = document.createElement('script');
        script.src = 'https://checkout.razorpay.com/v1/checkout.js';
        script.async = true;
        script.onload = () => {
            console.log('Razorpay script loaded successfully');
            setRazorpayLoaded(true);
        };
        script.onerror = () => {
            console.error('Failed to load Razorpay script');
            setMessage('Payment gateway failed to load. Please refresh the page and try again.');
            setMessageType('error');
        };
        document.body.appendChild(script);
        razorpayScriptRef.current = script;
    } else {
        setRazorpayLoaded(true);
    }

    return () => {
        // Cleanup script on component unmount
        if (razorpayScriptRef.current) {
            document.body.removeChild(razorpayScriptRef.current);
        }
    };
}, []);

// Fetch user credits on component mount
useEffect(() => {
    const fetchUserCredits = async () => {
        try {
            const token = await getToken();
            const response = await axios.get(apiEndpoints.GET_CREDITS, {
                headers: {
                    'Authorization': `Bearer ${token}`
                }
            });
            setCredits(response.data.credits);
        } catch (error) {
            console.error("Error fetching user credits:", error);
            setMessage("Failed to load your current credits. Please try again later.");
            setMessageType("error");
        }
    };

    fetchUserCredits();
```

```
}, [getToken]);

const handlePurchase = async (plan) => {
  if (!razorpayLoaded) {
    setMessage('Payment gateway is still loading. Please wait a moment and try again.');
    setMessageType('error');
    return;
  }

  setProcessingPayment(true);
  setMessage('');

  try {
    const token = await getToken();
    const response = await axios.post(apiEndpoints.CREATE_ORDER, {
      planId: plan.id,
      amount: plan.price * 100, // Razorpay expects amount in paise
      currency: "INR",
      credits: plan.credits
    }, {
      headers: {
        'Authorization': `Bearer ${token}`
      }
    });

    const options = {
      key: import.meta.env.VITE_RAZORPAY_KEY,
      amount: plan.price * 100,
      currency: "INR",
      name: "CloudShare",
      description: `Purchase ${plan.credits} credits`,
      order_id: response.data.orderId,
      handler: async function (response) {
        try {
          const verifyResponse = await axios.post(apiEndpoints.VERIFY_PAYMENT, {
            razorpay_order_id: response.razorpay_order_id,
            razorpay_payment_id: response.razorpay_payment_id,
            razorpay_signature: response.razorpay_signature,
            planId: plan.id
          }, {
            headers: {
              'Authorization': `Bearer ${token}`
            }
          });

          if (verifyResponse.data.success) {
            // Update credits immediately with the value from the response
            if (verifyResponse.data.credits) {
              console.log('Updating credits to:', verifyResponse.data.credits);
              setCredits(verifyResponse.data.credits);
            } else {
```

```
                    // If credits not in response, fetch the latest credits from the server
                    console.log('Credits not in response, fetching latest credits');
                    await fetchUserCredits();
                }

                setMessage(`Payment successful! ${plan.name} plan activated.`);
                setMessageType("success");
            } else {
                setMessage("Payment verification failed. Please contact support.");
                setMessageType("error");
            }
        } catch (error) {
            console.error("Payment verification error:", error);
            setMessage("Payment verification failed. Please contact support.");
            setMessageType("error");
        }
      },
      prefill: {
        name: user.fullName,
        email: user.primaryEmailAddress
      },
      theme: {
        color: "#3B82F6"
      }
    };
    if (window.Razorpay) {
      const razorpay = new window.Razorpay(options);
      razorpay.open();
    } else {
      throw new Error('Razorpay SDK not loaded');
    }
  }catch(error) {
    console.error("Payment initiation error:", error);
    setMessage("Failed to initiate payment. Please try again later.");
    setMessageType("error");
  }finally {
    setProcessingPayment(false);
  }
}

return (
  <DashboardLayout activeMenu="Subscription">
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-2">Subscription Plans</h1>
      <p className="text-gray-600 mb-6">Choose a plan that works for you</p>

      {message && (
        <div className={`mb-6 p-4 rounded-lg flex items-center gap-3 ${
          messageType === 'error' ? 'bg-red-50 text-red-700' :
            messageType === 'success' ? 'bg-green-50 text-green-700' :
              'bg-blue-50 text-blue-700'
```

```
            }`}>
              {messageType === 'error' && <AlertCircle size={20} />}
              {message}
            </div>
          )}

          <div className="flex flex-col md:flex-row gap-6 mb-8">
            <div className="bg-blue-50 p-6 rounded-lg">
              <div className="flex items-center gap-2 mb-4">
                <CreditCard className="text-purple-500" />
                <h2 className="text-lg font-medium">Current Credits: <span
```
```
className="font-bold text-purple-500">{credits}</span></h2>
              </div>
              <p className="text-sm text-gray-600 mt-2">
                You can upload {credits} more files with your current credits.
              </p>
            </div>
          </div>

          <div className="grid md:grid-cols-2 gap-6">
            {plans.map((plan) => (
              <div
                key={plan.id}
                className={`border rounded-xl p-6 ${
                  plan.recommended
                    ? 'border-purple-200 bg-purple-50 shadow-md'
                    : 'border-gray-200 bg-white'
                }`}
              >
                {plan.recommended && (
                  <div className="inline-block bg-purple-500 text-white text-xs font-semibold
```
```
px-3 py-1 rounded-full mb-4">
                    RECOMMENDED
                  </div>
                )}
                <h3 className="text-xl font-bold">{plan.name}</h3>
                <div className="mt-2 mb-4">
                  <span className="text-3xl font-bold">₹{plan.price}</span>
                  <span className="text-gray-500"> for {plan.credits} credits</span>
                </div>

                <ul className="space-y-3 mb-6">
                  {plan.features.map((feature, index) => (
                    <li key={index} className="flex items-start">
                      <Check size={18} className="text-green-500 mr-2 mt-0.5 flex-shrink-
```
```
0" />
                      <span>{feature}</span>
                    </li>
                  ))}
                </ul>
```

```
                    <button
                        onClick={() => handlePurchase(plan)}
                        disabled={processingPayment}
                        className={`w-full py-2 rounded-md font-medium transition-colors ${
                            plan.recommended
                                ? 'bg-purple-500 text-white hover:bg-purple-600'
                                : 'bg-white border border-purple-500 text-purple-500 hover:bg-purple-
50'
                        } disabled:opacity-50 flex items-center justify-center gap-2`}
                    >
                        {processingPayment ? (
                            <>
                                <Loader2 size={16} className="animate-spin" />
                                <span>Processing...</span>
                            </>
                        ) : (
                            <span>Purchase Plan</span>
                        )}
                    </button>
                </div>
            ))}
        </div>

        <div className="mt-8 bg-gray-50 p-4 rounded-lg border border-gray-200">
            <h3 className="font-medium mb-2">How credits work</h3>
            <p className="text-sm text-gray-600">
                Each file upload consumes 1 credit. New users start with 5 free credits.
                Credits never expire and can be used at any time. If you run out of credits,
                you can purchase more through one of our plans above.
            </p>
        </div>

    </div>
  </DashboardLayout>
 )
}

export default Subscription;
```

Transactions.jsx:

```
import DashboardLayout from "../layout/DashboardLayout.jsx";
import {useEffect, useState} from "react";
import {useAuth} from "@clerk/clerk-react";
import axios from "axios";
import {apiEndpoints} from "../util/apiEndpoints.js";
import {AlertCircle, Loader2, Receipt} from "lucide-react";

const Transactions = () => {
    const [transactions, setTransactions] = useState([]);
    const [loading, setLoading] = useState(false);
    const [error, setError] = useState(null);
```

```
const {getToken} = useAuth();

useEffect(() => {
   const fetchTransactions = async () => {
      try {
         setLoading(true);
         const token = await getToken();
         const response = await axios.get(
            apiEndpoints.TRANSACTIONS,
            {
               headers: {
                  Authorization: `Bearer ${token}`,
               },
            }
         );
         setTransactions(response.data);
         setError(null);
      } catch (error) {
         console.error("Error fetching transactions:", error);
         setError(
            "Failed to load your transaction history. Please try again later."
         );
      } finally {
         setLoading(false);
      }
   };
   fetchTransactions();
}, [getToken]);

const formatDate = (dateString) => {
   const options = {
      year: "numeric",
      month: "long",
      day: "numeric",
      hour: "2-digit",
      minute: "2-digit",
   };
   return new Date(dateString).toLocaleDateString(undefined, options);
};

// Format amount from paise to rupees
const formatAmount = (amountInPaise) => {
   return `₹${(amountInPaise / 100).toFixed(2)}`;
};

return (
   <DashboardLayout activeMenu="Transactions">
      <div className="p-6">
         <div className="flex items-center gap-2 mb-6">
            <Receipt className="text-blue-600" />
            <h1 className="text-2xl font-bold">Transaction History</h1>
```

```
        </div>

        {error && (
          <div className="mb-6 p-4 bg-red-50 text-red-700 rounded-lg flex items-center gap-
2">
            <AlertCircle size={20} />
            <span>{error}</span>
          </div>
        )}

        {loading ? (
          <div className="flex justify-center items-center h-64">
            <Loader2 className="animate-spin mr-2" size={24} />
            <span>Loading transactions...</span>
          </div>
        ): transactions.length === 0 ? (
          <div className="bg-gray-50 p-8 rounded-lg text-center">
            <Receipt size={48} className="mx-auto mb-4 text-gray-400" />
            <h3 className="text-lg font-medium text-gray-700 mb-2">
              No Transactions Yet
            </h3>
            <p className="text-gray-500">
              You haven't made any credit purchases yet. Visit the Subscription
              page to buy credits.
            </p>
          </div>
        ): (
          <div className="overflow-x-auto">
            <table className="min-w-full bg-white rounded-lg overflow-hidden shadow">
              <thead className="bg-gray-50">
                <tr>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">
                    Date
                  </th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">
                    Plan
                  </th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">
                    Amount
                  </th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">
                    Credits Added
                  </th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase tracking-wider">
                    Payment ID
                  </th>
```

```
                    </tr>
                  </thead>
                  <tbody className="divide-y divide-gray-200">
                  {transactions.map((transaction) => (
                    <tr key={transaction.id} className="hover:bg-gray-50">
                      <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-900">
                        {formatDate(transaction.transactionDate)}
                      </td>
                      <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-900">
                        {transaction.planId === "premium"
                          ? "Premium Plan"
                          : transaction.planId === "ultimate"
                            ? "Ultimate Plan"
                            : "Basic Plan"}
                      </td>
                      <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-900">
                        {formatAmount(transaction.amount)}
                      </td>
                      <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-900">
                        {transaction.creditsAdded}
                      </td>
                      <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-500 font-
mono">
                        {transaction.paymentId
                          ? transaction.paymentId.substring(0, 12) + "..."
                          : "N/A"}
                      </td>
                    </tr>
                  ))}
                  </tbody>
                </table>
              </div>
            )}
          </div>
        </DashboardLayout>
    )
}

export default Transactions;
```

Upload.jsx:

```
import DashboardLayout from "../layout/DashboardLayout.jsx";
import {useContext, useState} from "react";
import {useAuth} from "@clerk/clerk-react";
import {UserCreditsContext} from "../context/UserCreditsContext.jsx";
import {AlertCircle} from "lucide-react";
import axios from "axios";
import {apiEndpoints} from "../util/apiEndpoints.js";
import UploadBox from "../components/UploadBox.jsx";

const Upload = () => {
```

```
const [files, setFiles] = useState([]);
const [uploading, setUploading] = useState(false);
const [message, setMessage] = useState("");
const [messageType, setMessageType] = useState(""); //success or error
const {getToken} = useAuth();
const {credits, setCredits} = useContext(UserCreditsContext);
const MAX_FILES = 5;

const handleFileChange = (e) => {
   const selectedFiles = Array.from(e.target.files);

   if (files.length + selectedFiles.length > MAX_FILES) {
      setMessage(`You can only upload a maximum of ${MAX_FILES} files at once`);
      setMessageType("error");
      return;
   }

   //add the new files into the existing files
   setFiles((prevFiles) => [...prevFiles, ...selectedFiles]);
   setMessage("");
   setMessageType("");
}

const handleRemoveFile = (index) => {
   setFiles((prevFiles) => prevFiles.filter((_, i) => i !== index));
   setMessageType("");
   setMessage("");
}

const handleUpload = async () => {
   if (files.length === 0){
      setMessageType("error");
      setMessage("Please select atleast one file to upload.");
      return;
   }

   if (files.length > MAX_FILES) {
      setMessage(`You can only upload a maximum of ${MAX_FILES} files at once.`);
      setMessageType("error");
      return;
   }

   setUploading(true);
   setMessage("Uploading files...");
   setMessageType("info");

   const formData = new FormData();
   files.forEach((file) => formData.append("files", file));

   try {
      const token = await getToken();
```

```
        const response = await axios.post(apiEndpoints.UPLOAD_FILE, formData, {headers:
{"Content-Type": "multipart/form-data", Authorization: `Bearer ${token}`}});

        if (response.data && response.data.remainingCredits !== undefined) {
          setCredits(response.data.remainingCredits);
        }

        setMessage("Files uploaded successfully.");
        setMessageType("success");
        setFiles([]);
      }catch(error) {
        console.error('Error uploading files: ', error);
        setMessage(error.response?.data?.message || "Error uploading files. Please try again.");
        setMessageType("error");
      }finally {
        setUploading(false);
      }
    }

    const isUploadDisabled = files.length === 0 || files.length > MAX_FILES || credits <= 0 ||
files.length > credits;

    return (
      <DashboardLayout activeMenu="Upload">
        <div className="p-6">
          {message && (
            <div className={`mb-6 p-4 rounded-lg flex items-center gap-3 ${messageType ===
'error' ? 'bg-red-50 text-red-700': messageType === 'success' ? 'bg-green-50 text-green-700': 'bg-
blue-50 text-blue-700'}`}>
              {messageType === 'error' && <AlertCircle size={20} />}
              {message}
            </div>
          )}

          <UploadBox
            files={files}
            onFileChange={handleFileChange}
            onUpload={handleUpload}
            uploading={uploading}
            onRemoveFile={handleRemoveFile}
            remainingCredits={credits}
            isUploadDisabled={isUploadDisabled}
          />
        </div>
      </DashboardLayout>
    )
}

export default Upload;
```

(Backend)
CloudstorageApplication.java:

```
package in.vedutech.cloudstorage;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CloudstorageApplication {

   public static void main(String[] args) {
     SpringApplication.run(CloudstorageApplication.class, args);
   }

}
```

SecurityConfig.java:

```
package in.vedutech.cloudstorage.config;
import in.vedutech.cloudstorage.security.ClerkJwtAuthFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;

import java.util.List;

@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

   private final ClerkJwtAuthFilter clerkJwtAuthFilter;

   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {
      httpSecurity.cors(Customizer.withDefaults())
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(auth -> auth.requestMatchers("/webhooks/**",
"/files/**").permitAll().anyRequest().authenticated())
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(clerkJwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

```
        return httpSecurity.build();
    }

    @Bean
    public CorsFilter corsFilter() {
        return new CorsFilter(corsConfigurationSource());
    }

    private UrlBasedCorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of("*"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE",
"OPTIONS"));
        config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        config.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}
```

StaticResourceConfig.java:

```
package in.vedutech.cloudstorage.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import java.nio.file.Path;
import java.nio.file.Paths;

@Configuration
public class StaticResourceConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        String uploadDir = Paths.get("uploads").toAbsolutePath().toString();
        registry.addResourceHandler("/uploads/**")
                .addResourceLocations("file:" + uploadDir + "/");
    }
}
```

FileMetadataDocument.java:

```
package in.vedutech.cloudstorage.document;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
```

```
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Document(collection = "files")
@AllArgsConstructor
@NoArgsConstructor
@Data
@Builder
public class FileMetadataDocument {

    @Id
    private String id;
    private String name;
    private String type;
    private Long size;
    private String clerkId;
    private Boolean isPublic;
    private String fileLocation;
    private LocalDateTime uploadedAt;
}
```

ProfileDocument.java:

```
package in.vedutech.cloudstorage.document;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.Instant;

@AllArgsConstructor
@NoArgsConstructor
@Data
@Builder
@Document(collection = "profiles")

public class ProfileDocument {
    @Id
    @CreatedDate
    private String id;
    private String clerkId;
    @Indexed(unique = true)
    private String email;
    private String firstName;
```

```
    private String lastName;
    private Integer credits;
    private String photoUrl;
    private Instant createdAt;
}
```

UserCredits.java:

```
package in.vedutech.cloudstorage.document;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "user_credits")
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class UserCredits {
    @Id
    private String id;
    private String clerkId;
    private Integer credits;
    private String plan;

}
```

ClerkJwksProvider.java:

```
package in.vedutech.cloudstorage.security;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.math.BigInteger;
import java.net.URL;
import java.security.KeyFactory;
import java.security.PublicKey;
import java.security.spec.RSAPublicKeySpec;
import java.util.Base64;
import java.util.HashMap;
import java.util.Map;

@Component
public class ClerkJwksProvider {

    @Value("${clerk.jwks-url}")
    private String jwksUrl;
```

```
    private final Map<String, PublicKey> keyCache = new HashMap<>();
    private long lastFetchTime = 0;
    private static final long CACHE_TTL = 3600000; //1 hour

    public PublicKey getPublicKey(String kid) throws Exception{
        if (keyCache.containsKey(kid) && System.currentTimeMillis() - lastFetchTime <
CACHE_TTL) {
            return keyCache.get(kid);
        }

        refreshKeys();
        return keyCache.get(kid);
    }

    private void refreshKeys() throws Exception{
        ObjectMapper mapper = new ObjectMapper();
        JsonNode jwks = mapper.readTree(new URL(jwksUrl));

        JsonNode keys = jwks.get("keys");
        for(JsonNode keyNode: keys) {
            String kid = keyNode.get("kid").asText();
            String kty = keyNode.get("kty").asText();
            String alg = keyNode.get("alg").asText();

            if ("RSA".equals(kty) && "RS256".equals(alg)) {
                String n = keyNode.get("n").asText();
                String e = keyNode.get("e").asText();

                PublicKey publicKey = createPublicKey(n, e);
                keyCache.put(kid, publicKey);
            }
        }
        lastFetchTime = System.currentTimeMillis();
    }

    private PublicKey createPublicKey(String modulus, String exponent) throws Exception {
        byte[] modulusBytes = Base64.getUrlDecoder().decode(modulus);
        byte[] exponentBytes = Base64.getUrlDecoder().decode(exponent);

        BigInteger modulusBigInt = new BigInteger(1, modulusBytes);
        BigInteger exponentBigInt = new BigInteger(1, exponentBytes);

        RSAPublicKeySpec spec = new RSAPublicKeySpec(modulusBigInt, exponentBigInt);
        KeyFactory factory = KeyFactory.getInstance("RSA");
        return factory.generatePublic(spec);
    }
}
```

ClerkJwtAuthFilter.java:

```
package in.vedutech.cloudstorage.security;
```

```
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.security.PublicKey;
import java.util.Base64;
import java.util.Collections;

@Component
@RequiredArgsConstructor
public class ClerkJwtAuthFilter extends OncePerRequestFilter {

    @Value("${clerk.issuer}")
    private String clerkIssuer;

    private final ClerkJwksProvider jwksProvider;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain) throws ServletException, IOException {

        if (request.getRequestURI().contains("/webhooks")) {
            filterChain.doFilter(request, response);
        }

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null || !authHeader.startsWith("Bearer ")) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Authorization header
missing/invalid");
            return;
        }

        try {
            String token = authHeader.substring(7);
            String[] chunks = token.split("\\.");
            if (chunks.length < 3) {
```

```
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Invalid JWT token
format");
            return;
        }

        String headerJson = new String(Base64.getUrlDecoder().decode(chunks[0]));
        ObjectMapper mapper = new ObjectMapper();
        JsonNode headerNode = mapper.readTree(headerJson);

        if (!headerNode.has("kid")) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Token header is missing
kid");
            return;
        }

        String kid = headerNode.get("kid").asText();

        PublicKey publicKey = jwksProvider.getPublicKey(kid);

        //verify the token
        Claims claims = Jwts.parserBuilder()
                .setSigningKey(publicKey)
                .setAllowedClockSkewSeconds(60)
                .requireIssuer(clerkIssuer)
                .build()
                .parseClaimsJws(token)
                .getBody();

        String clerkId = claims.getSubject();

        UsernamePasswordAuthenticationToken authenticationToken = new
UsernamePasswordAuthenticationToken(clerkId, null, Collections.singletonList(new
SimpleGrantedAuthority("ROLE_ADMIN")));

        SecurityContextHolder.getContext().setAuthentication(authenticationToken);
        filterChain.doFilter(request, response);
    } catch (Exception e) {
        response.sendError(HttpServletResponse.SC_FORBIDDEN, "Invalid JWT token:
"+e.getMessage());
        return;
    }


  }
}
```

5.2.2 Code Efficiency

Code efficiency was ensured by adopting best practices in both frontend and backend development. In the frontend, React's component reusability minimized redundant code and improved rendering performance through the Virtual DOM. API calls were optimized to fetch only required data, reducing unnecessary network traffic. Proper state management avoided excessive re-rendering, improving overall user experience.

On the backend, Spring Boot's dependency injection and modular architecture improved code maintainability and execution speed. MongoDB's document-based structure allowed efficient data retrieval without complex joins, enhancing database performance. JWT-based stateless authentication reduced server memory usage, as no session data was stored on the server. Indexing was applied in MongoDB for frequently accessed fields such as userId and email to improve query performance.

Error handling and validation logic were structured efficiently to avoid application crashes and unnecessary database queries. The system was tested for optimized API response time, ensuring smooth interaction between frontend and backend. Overall, the coding approach focused on clean architecture, reduced redundancy, optimized database queries, and secure API communication, resulting in a stable and efficient digital vault system.

5.3 Testing Approaches

5.3.1 Unit Testing

Unit Testing is a software testing technique in which individual components or units of the system are tested independently to ensure that each unit functions correctly. A unit can be a small piece of code such as a function, method, or module. The main objective of unit testing is to verify that each part of the application performs as expected before integrating it with other components.

In the Nominee-Based Access System, unit testing was performed during the development phase to validate individual modules such as user authentication, nominee management, document upload, and API request handling. Each module was tested in isolation to ensure that the logic, input validation, and output responses were working correctly. This approach helped in identifying and fixing errors at an early stage, reducing the chances of defects in later stages of development.

Unit testing was mainly carried out by developers using predefined test cases. For example, authentication functions were tested to verify correct login behavior, token generation, and error handling for invalid credentials. Similarly, backend APIs were tested to ensure proper data processing and response generation.

The use of unit testing improved code reliability, simplified debugging, and ensured that every component met its functional requirements. It also made the system more maintainable and scalable by ensuring that changes in one module did not affect others.

5.3.2 Integration Testing

Integration Testing is a testing technique in which different modules of the system are combined and tested together to ensure that they work properly as a group. After performing unit testing on individual components, integration testing is done to verify the interaction between those components.

In simple terms, once we confirm that each part is working correctly, we check how these parts communicate with each other.

In my project (Nominee-Based Access System), integration testing was used to test how the frontend, backend, authentication system, and database work together. For example, I tested whether the React frontend correctly sends requests to the backend APIs and whether the backend properly processes the request and retrieves data from MongoDB.

I also checked the integration of authentication using Clerk and JWT tokens to ensure that only authorized users can access protected routes and APIs. This helped in verifying that the data flow between different modules was secure and accurate.

Integration testing helped in identifying issues related to data flow, API communication, and module interaction. It ensured that all parts of the system were properly connected and working together as expected.

5.3.3 Beta Testing

Beta Testing is a type of testing in which the software is given to a limited number of real users outside the development team to test it in a real-world environment. It is usually performed after the system testing phase and before the final release of the product.

In simple terms, beta testing means allowing actual users to use the system and provide feedback based on their experience.

In my project (Nominee-Based Access System), beta testing was done by sharing the application with a small group of users. They were asked to perform tasks such as login, adding nominees, uploading documents, and accessing shared data. This helped in checking how the system performs in real-life scenarios.

During beta testing, feedback was collected from users regarding usability, performance, and any issues they faced while using the system. Some minor bugs and user interface improvements were identified and fixed based on this feedback.

Beta testing is important because it helps in identifying issues that may not be detected during development and internal testing. It ensures that the system is user-friendly, reliable, and ready for final deployment.

5.4 Modifications and Improvements

During the development of the Nominee-Based Access System, several modifications and improvements were made to enhance the performance, security, and usability of the system. Initially, the system had basic functionalities, but as development progressed, improvements were made based on testing results and user feedback. The user interface was enhanced using Tailwind CSS to make it more responsive and user-friendly across different devices. Security was significantly improved by implementing proper authentication and authorization mechanisms using Clerk and JWT tokens. Protected routes were added to ensure that only authorized users can access sensitive data. Additional validation checks were also implemented on both frontend and backend to prevent invalid inputs.

Performance improvements were made by optimizing API calls and reducing unnecessary data processing. Efficient data handling techniques were used in MongoDB to ensure faster retrieval and storage of data. Based on feedback received during testing phases such as beta testing, minor bugs were fixed and user experience was improved. Features like better error messages, smoother navigation, and improved document handling were added to make the system more reliable. Overall, these modifications and improvements helped in making the system more secure, efficient, and user-friendly, ensuring that it meets the project requirements effectively.

5.5 Test Cases

Test cases are used to verify whether the system is working as expected by checking different functionalities with predefined inputs and expected outputs. Each test case includes details such as the test scenario, input data, expected result, and actual result. In my project (Nominee-Based Access System), test cases were designed to validate major functionalities like user authentication, nominee management, document upload, and secure access control. These test cases helped in ensuring that the system behaves correctly under different conditions.

For example, in the login module, test cases were created to check both valid and invalid login attempts. Similarly, test cases were designed to verify whether users can successfully add nominees, upload documents, and access data based on permissions. Test cases also helped in identifying errors and ensuring that all features meet the required specifications. By executing these test cases, it became easier to track issues and confirm that the system is reliable and working correctly.

Test Case ID	Module	Test Scenario	Input Data	Expected Result	Actual Result	Status
TC01	Login	Valid Login	Correct email & password	User successfully logs in	As Expected	Pass
TC02	Login	Invalid Login	Wrong password	Error message displayed	As Expected	Pass
TC03	Nominee	Add Nominee	Valid nominee details	Nominee added successfully	As Expected	Pass
TC04	Document	Upload Document	Valid file	Document uploaded successfully	As Expected	Pass
TC05	Authentication	Access Protected Route	Without login	Access denied	As Expected	Pass

Chapter 6
Results and Discussion